

SE203 – Software Engineering

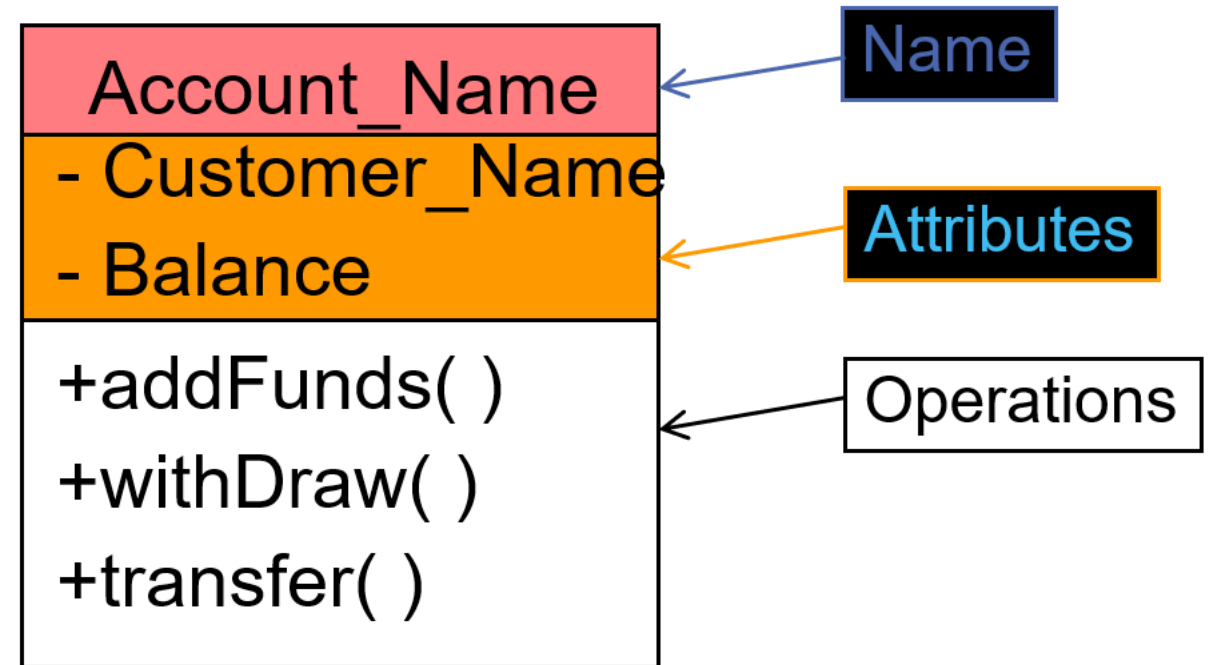
Structural Models
(Class, Object, Package, Composite,
Component, Deployment, DFD)

Class Diagrams

- A Class Diagram represents a system in terms of
 - *objects*: akin to entities, organized in classes that have an inheritance hierarchy
 - *attributes*: object's variables or characteristics
 - *methods*: actions on the object's variables
- A class diagram depicts classes and their interrelationships
- Used for describing structure and behavior in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers

Class Diagrams

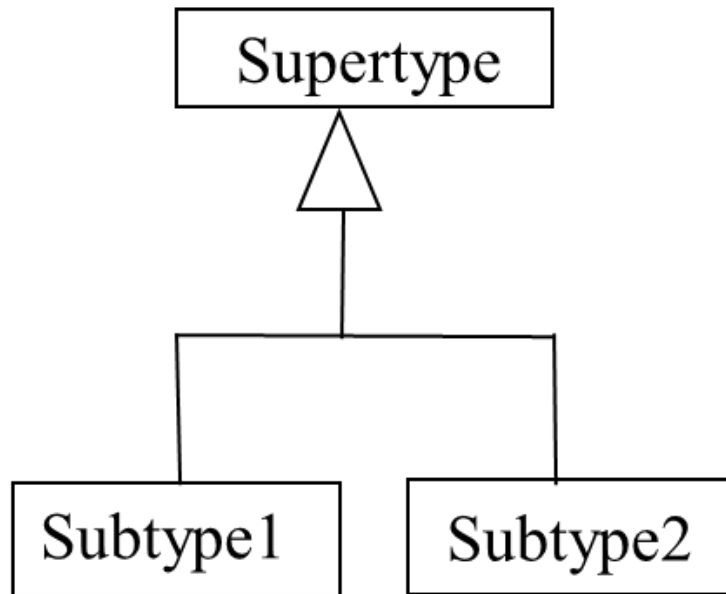
- Each class is represented by a rectangle subdivided into three compartments
 - Name
 - Attributes
 - Operations
- Modifiers are used to indicate visibility of attributes and operations.
 - '+' is used to denote *Public* visibility (everyone)
 - '#' is used to denote *Protected* visibility (friends and derived)
 - '-' is used to denote *Private* visibility (no one)
- By default, attributes are hidden and operations are visible.



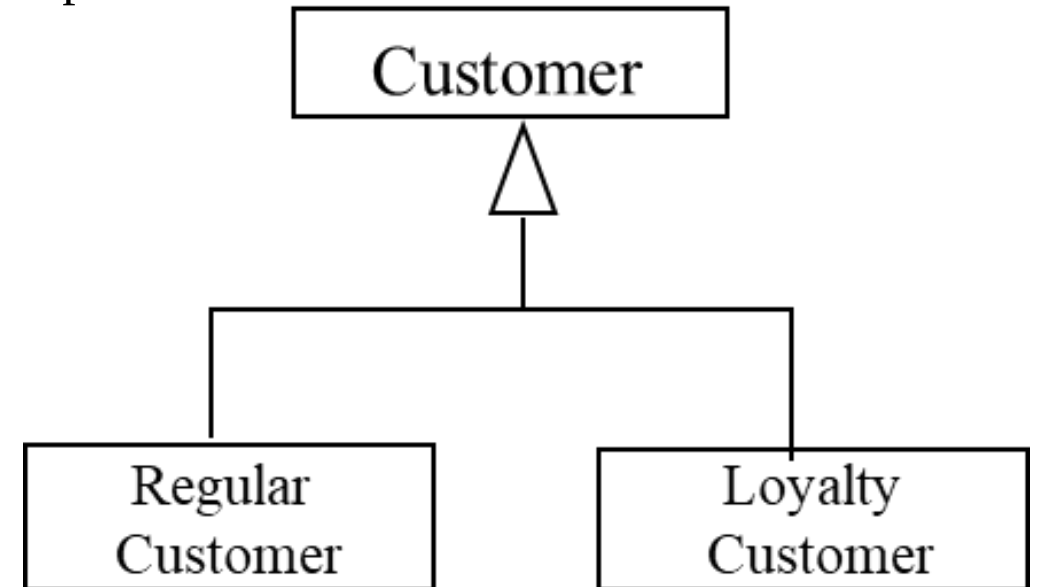
Class Diagrams Relationships

- There are two kinds of Relationships
 - Generalization (parent-child relationship)
 - Association (student enrolls in course)
- Associations can be further classified as
 - Aggregation
 - Composition

Generalization



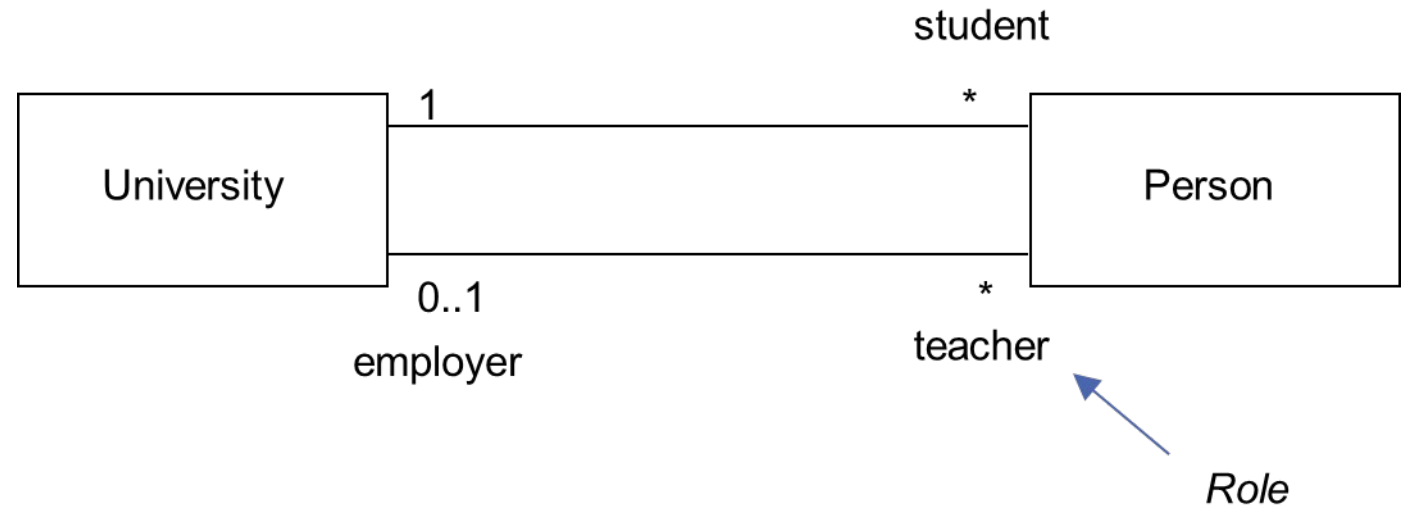
Example:



- Inheritance is a required feature of object orientation
- Generalization expresses a parent/child relationship among related classes.
- Used for abstracting details in several layers

Association

- Represent relationship between instances of classes
 - Student enrolls in a course
 - Courses have students
 - Courses have exams
 - Etc.
- Association has two ends
 - Role names (e.g. enrolls)
 - Multiplicity (e.g. One course can have many students)
 - Navigability (unidirectional, bidirectional)



Multiplicity	
<u>Symbol</u>	<u>Meaning</u>
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

“A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time.”

Composition

Association

Models the part–whole relationship

Composition

Also models the part–whole relationship but, in addition, Every part may belong to only one whole, and If the whole is deleted, so are the parts

Example:

A number of different chess boards:
Each square belongs to only one board. If a chess board is thrown away, all 64 squares on that board go as well.

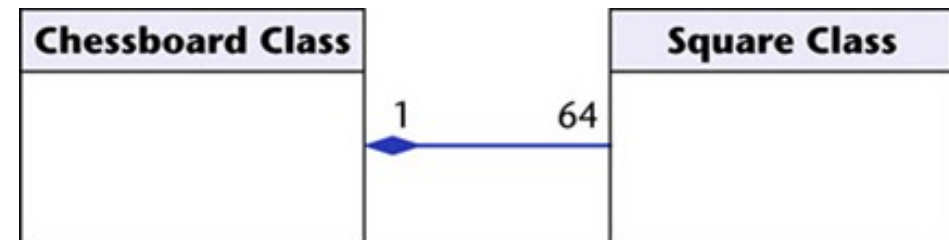
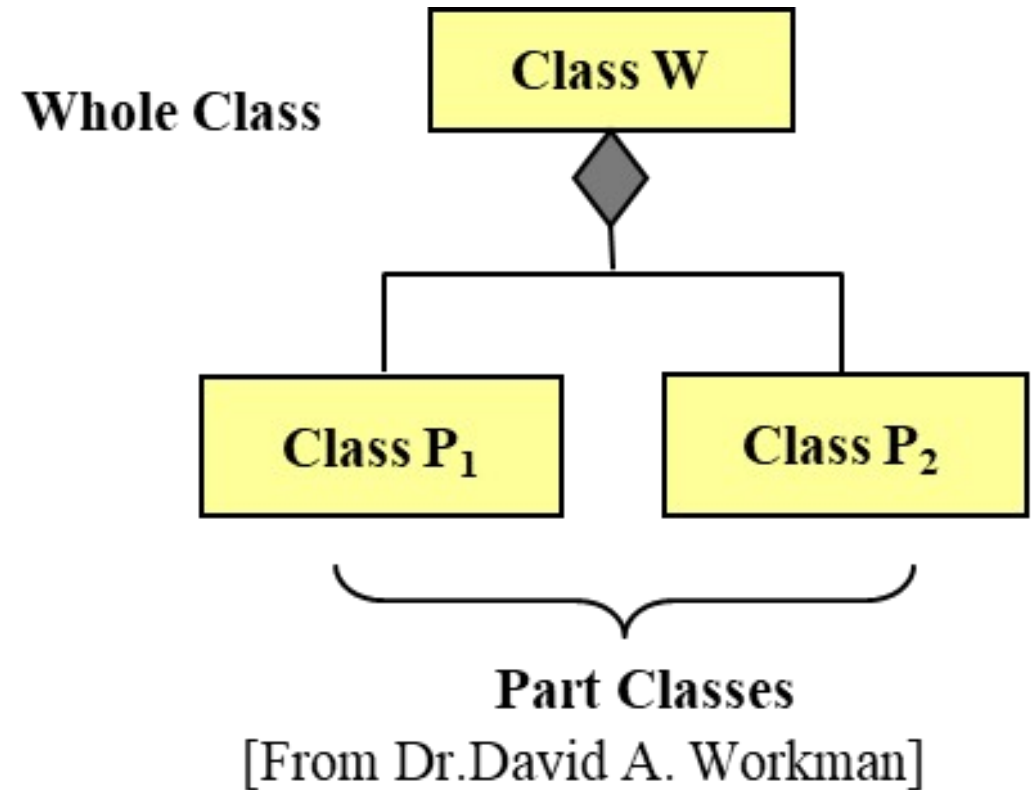


Figure 16.7

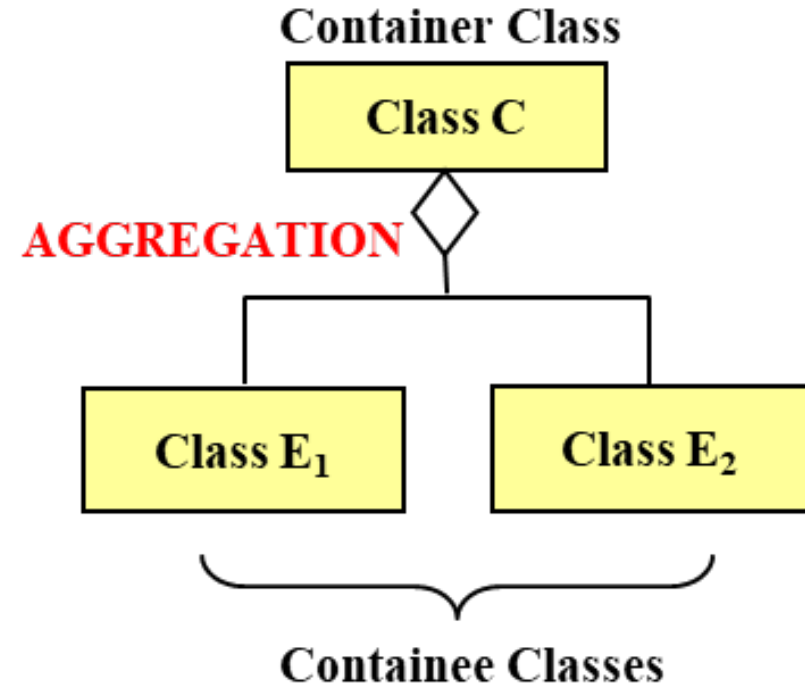
Aggregation

Aggregation

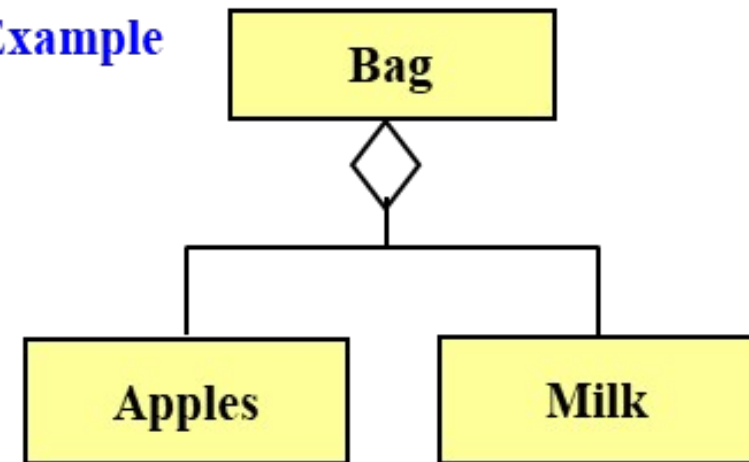
expresses a relationship among instances of related classes. It is a specific kind of Container-Containee relationship.

express a more informal relationship than composition expresses.

Aggregation is appropriate when Container and Containees have no special access privileges to each other.

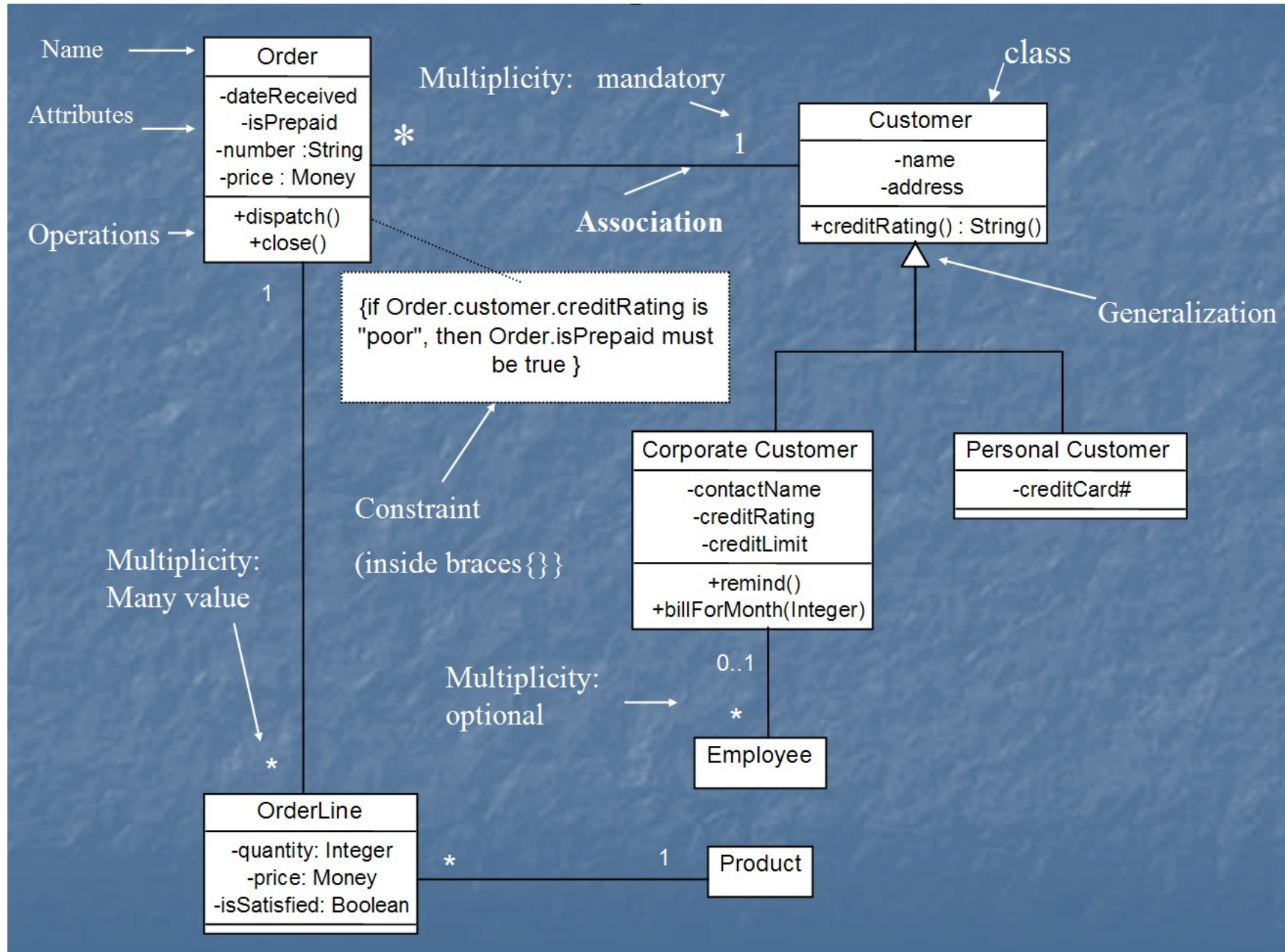


Example



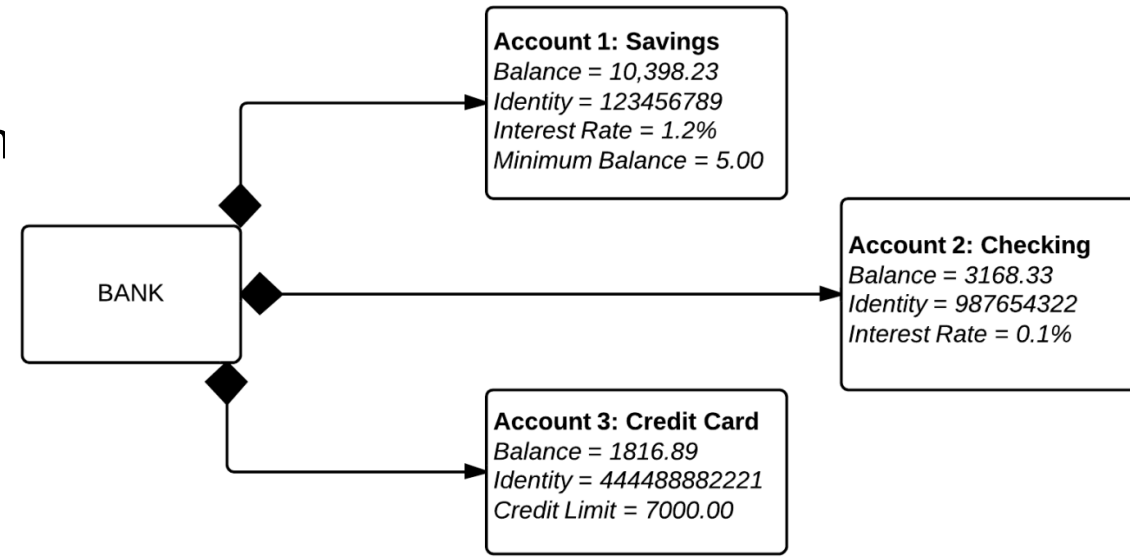
[From Dr.David A. Workman]

Class diagram



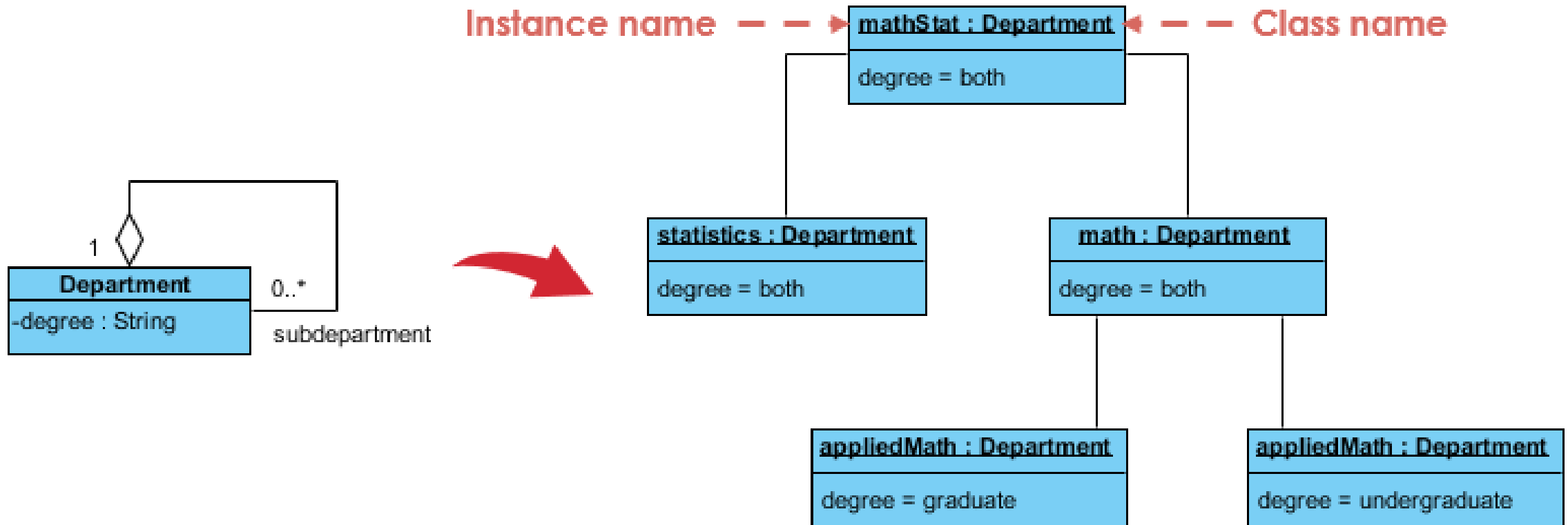
Object Diagram

- A UML object diagram represents a specific instance of a class diagram at a certain moment in time.
- An object diagram focuses on the **attributes** of a set of objects and how those objects relate to each other.
 - For instance, in this object diagram below, all three bank accounts tie back to the bank itself.
 - The class titles show the type of accounts (savings, checking, and credit card) that a given customer could have with this particular bank.
 - The class attributes are different for each account type. For example, the credit card object has a credit limit, while the savings and checking accounts have interest rates.



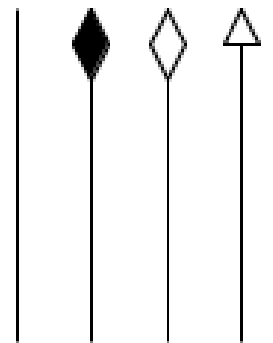
Class Diagram to Object Diagram

- This small class diagram shows that a university Department can contain lots of other Departments and the object diagram below instantiates the class diagram, replacing it by a concrete example.

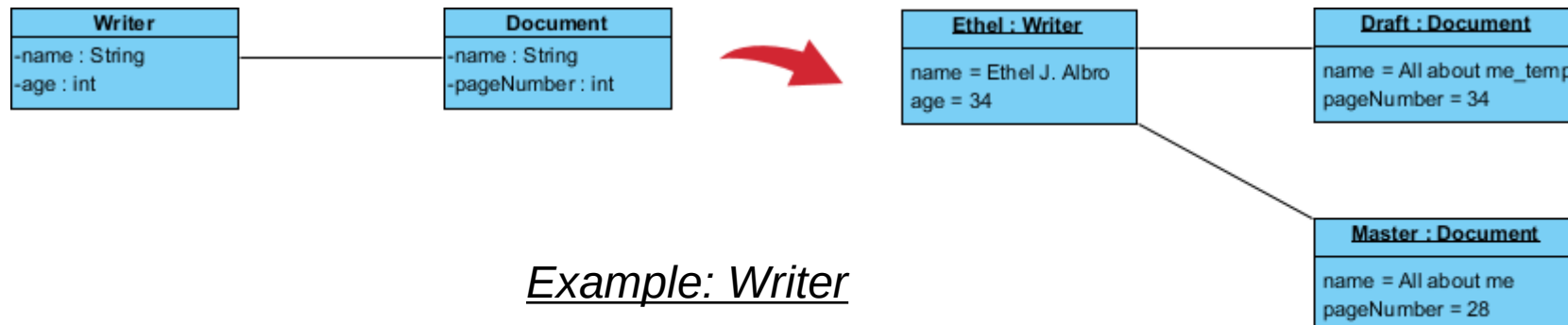
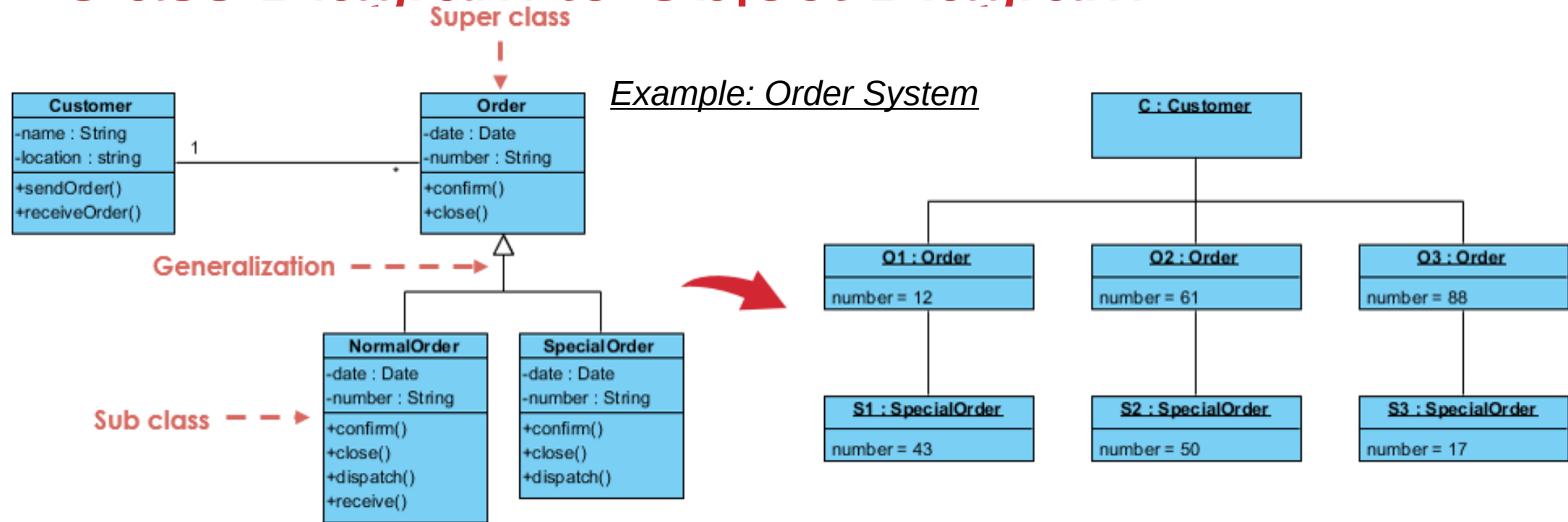


Basic Object Diagram Symbols and Notations

- Object Names:
- Every object is actually symbolized like a rectangle, that offers the name from the object and its class underlined as well as divided with a colon.
- Object Attributes:
- Similar to classes, you are able to list object attributes inside a separate compartment. However, unlike classes, object attributes should have values assigned for them.
- Links:
- Links tend to be instances associated with associations. You can draw a link while using the lines utilized in class diagrams.



Class Diagram to Object Diagram



Class Diagram

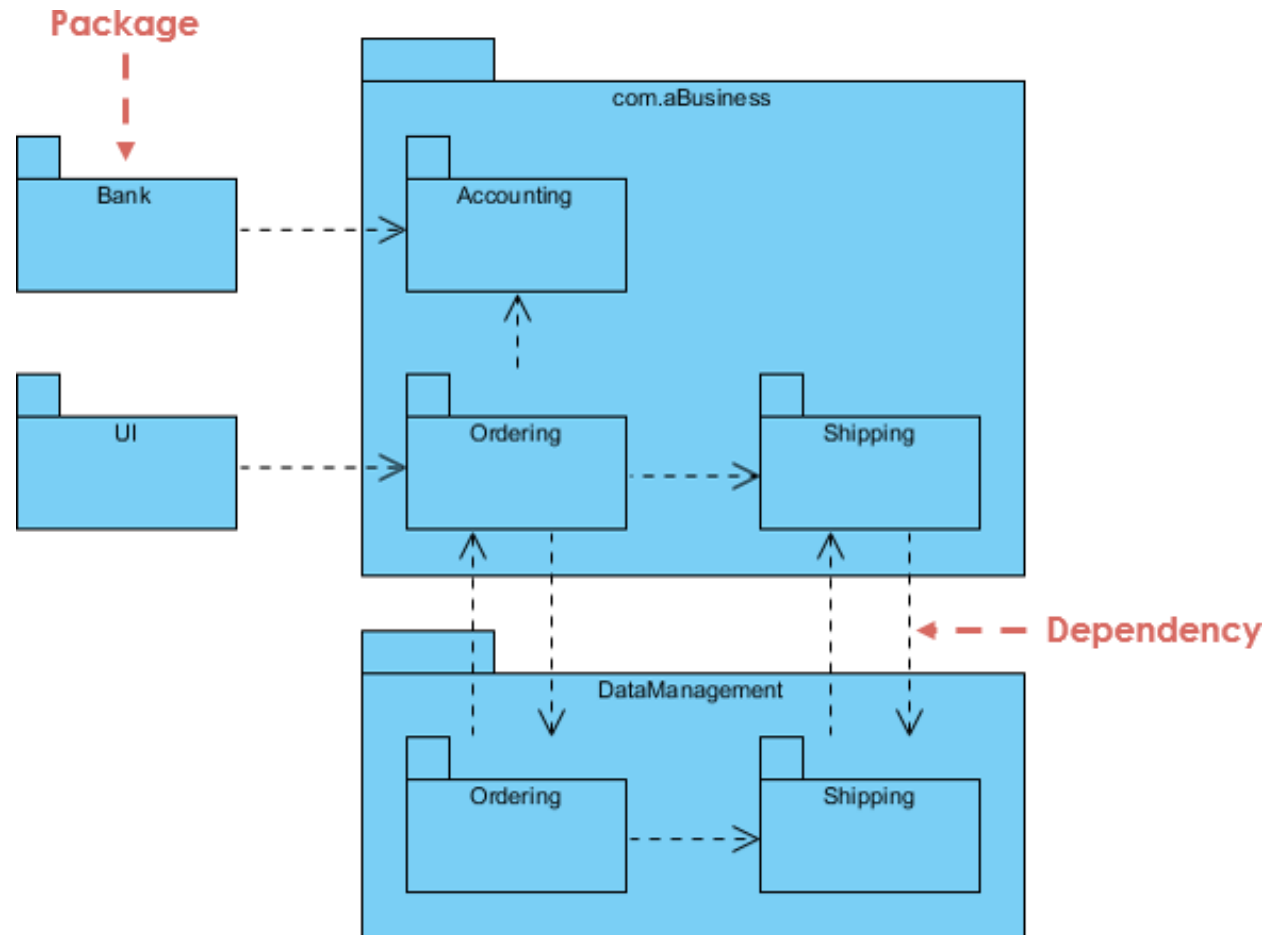
Object Diagram

PACKAGE Diagram

- Package diagram is used to simplify complex class diagrams, you can group classes into packages.
- Package Diagram can be used to simplify complex class diagrams, it can group classes into packages.
- A package is a collection of logically related UML elements.
- Packages are depicted as file folders and can be used on any of the UML diagrams.

PACKAGE Diagram

- The diagram is a business model in which the classes are grouped into packages:
- Packages appear as rectangles with small tabs at the top.
- The package name is on the tab or inside the rectangle.
- The dotted arrows are dependencies.
- One package depends on another if changes in the other could possibly force changes in the first.



Basic Concepts of Package Diagram

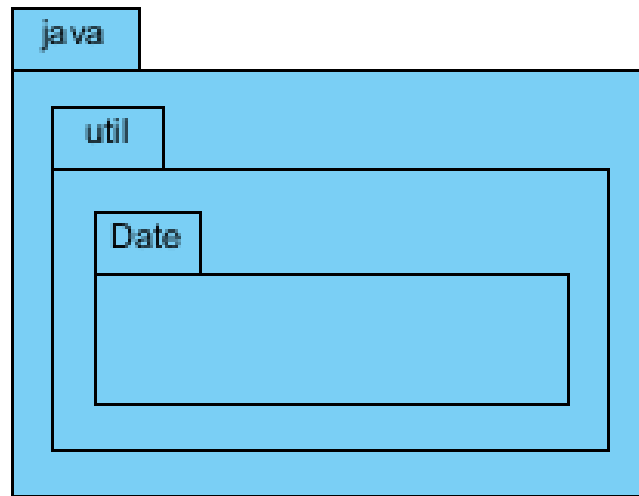
1. Package diagram follows hierarchal structure of nested packages. Atomic module for nested package are usually class diagrams. There are few constraints while using package diagrams, they are as follows.
2. Package name should not be the same for a system, however classes inside different packages could have the same name.
3. Packages can include whole diagrams, name of components alone or no components at all.
4. Fully qualified name of a package has the following syntax.

Name owing the package :: Name of the package

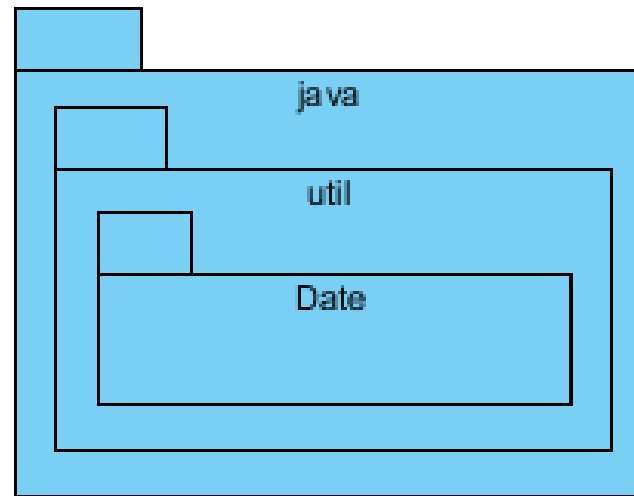
java :: util :: Date

Basic Concepts of Package Diagram

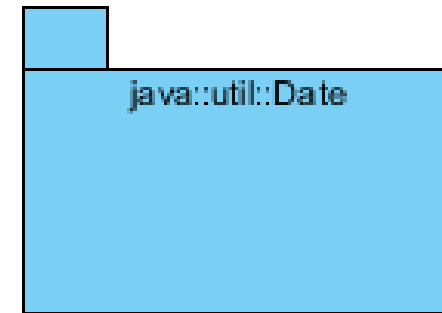
5. Packages can be represented by the notations with some examples shown below:



Nested, with captions in tab



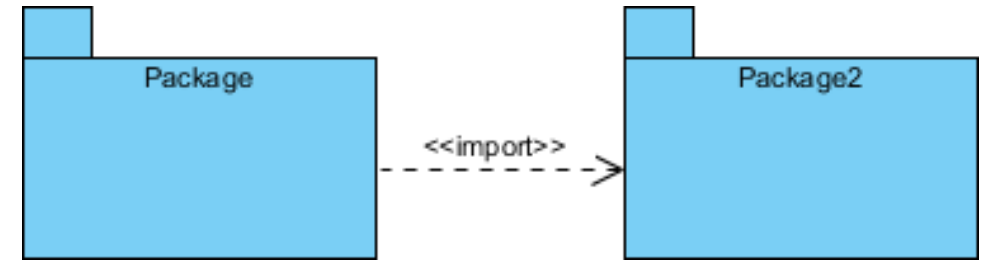
Nested, with captions in
package body



Fully qualified

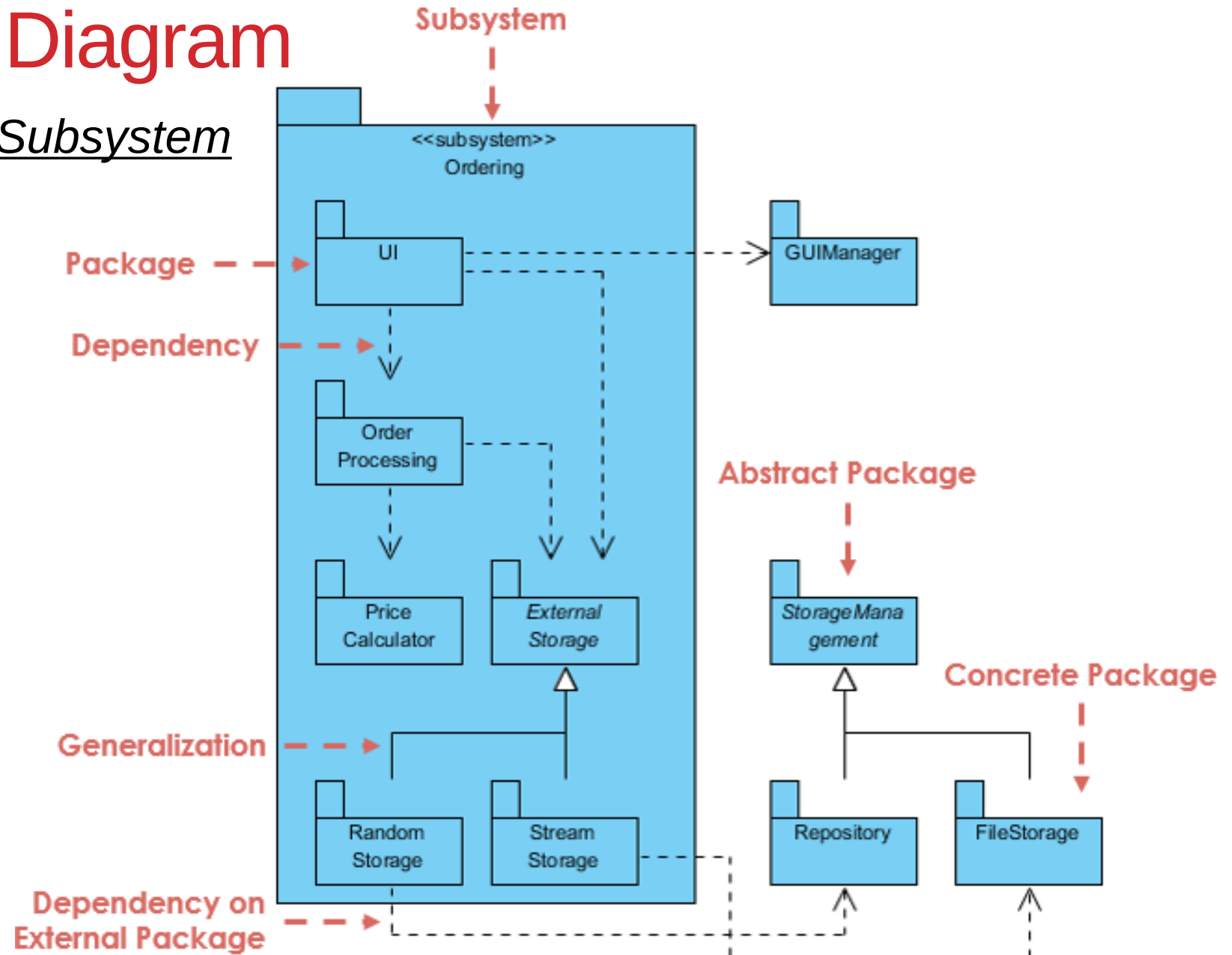
Package Diagram - Dependency Notation

- There are two sub-types involved in dependency.
- They are <<import>> & <<access>>.
- <<import>> - one package imports the functionality of other package.
- <<access>> - one package requires help from functions of other package.



PACKAGE Diagram

- Example: Order Subsystem

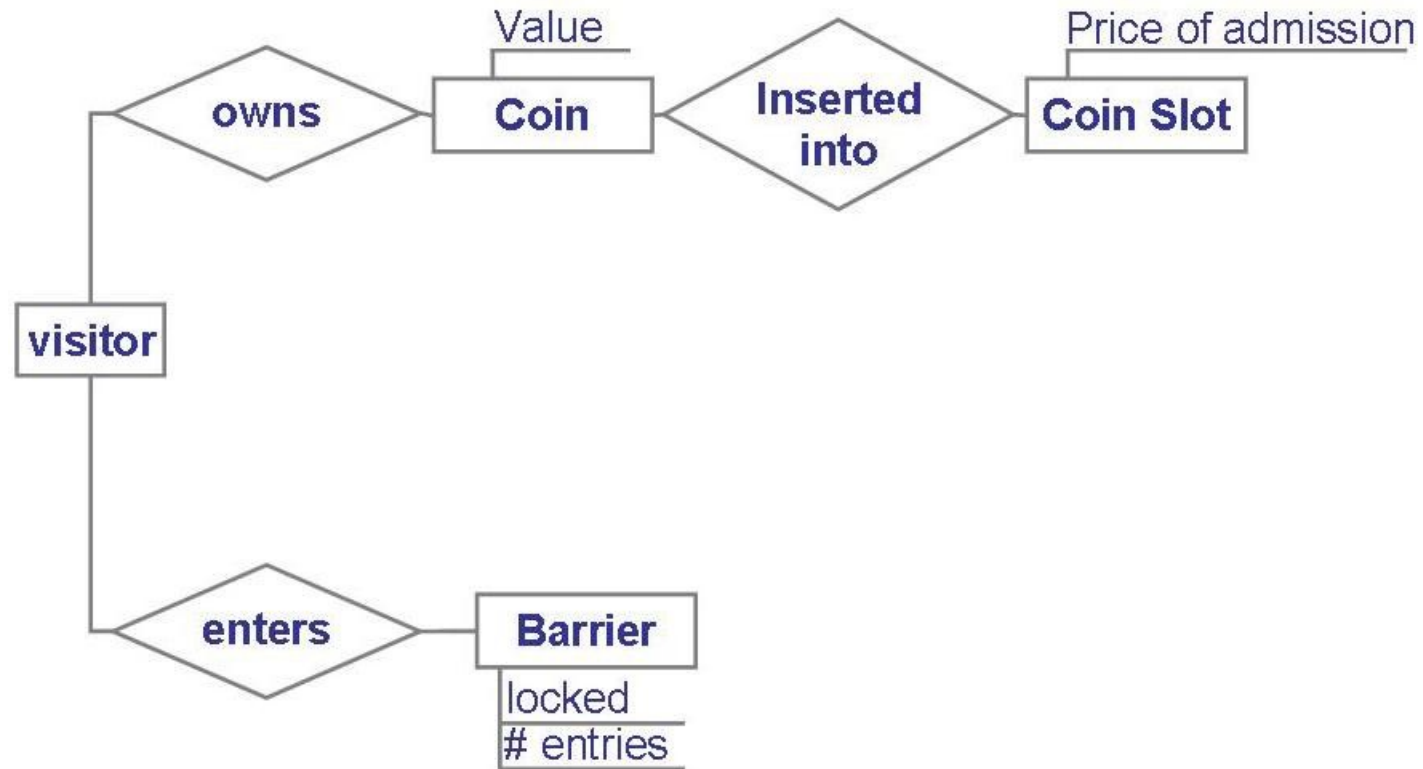


ENTITY-RELATIONSHIP DIAGRAMS

- A popular graphical notational paradigm for representing conceptual models
- Has three core constructs
 - An *entity*: depicted as a rectangle, represents a collection of real-world objects that have common properties and behaviors
 - A *relationship*: depicted as an edge between two entities, with diamond in the middle of the edge specifying the type of relationship
 - An *attribute*: an annotation on an entity that describes data or properties associated with the entity

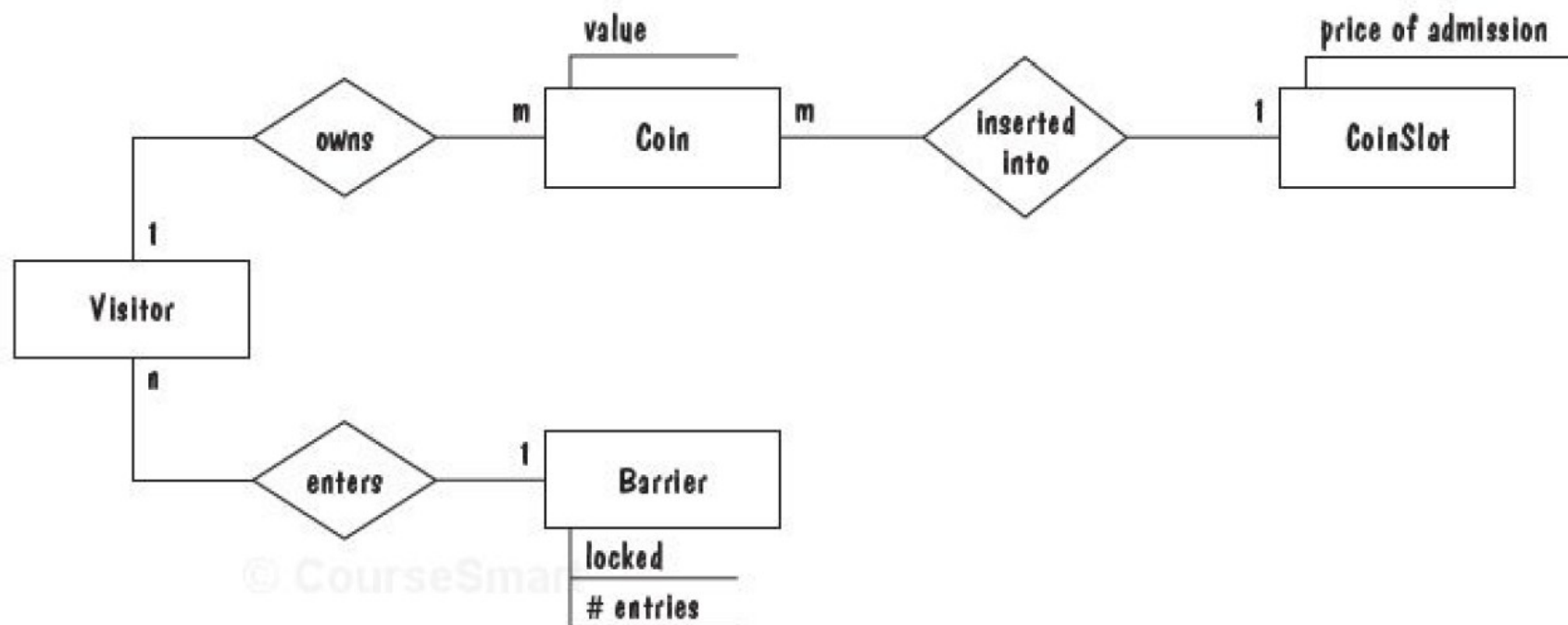
ENTITY-RELATIONSHIP DIAGRAMS

- Entity diagram of turnstile problem



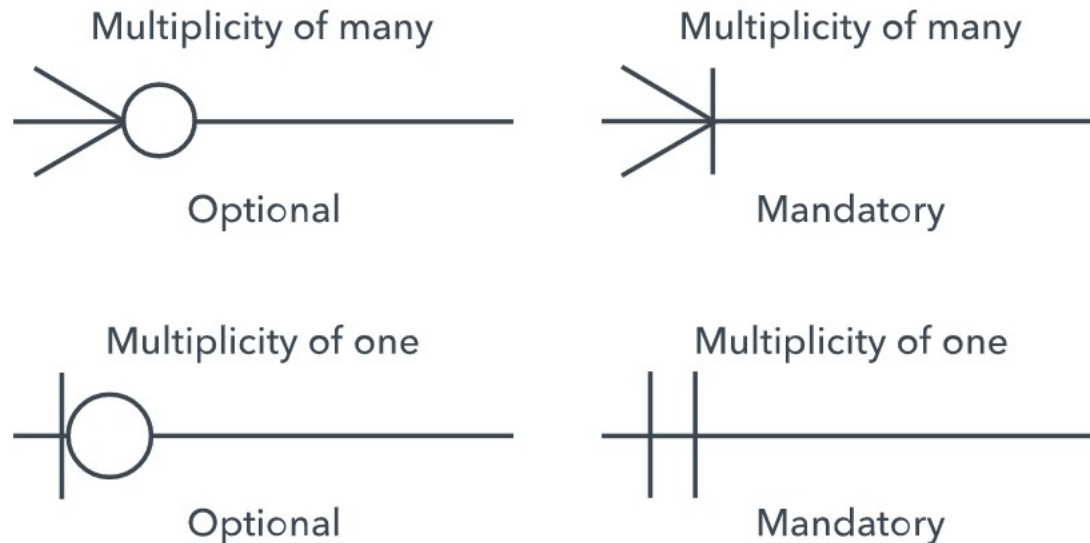
ENTITY-RELATIONSHIP DIAGRAMS

- Entity diagram of turnstile problem (with Cardinality)



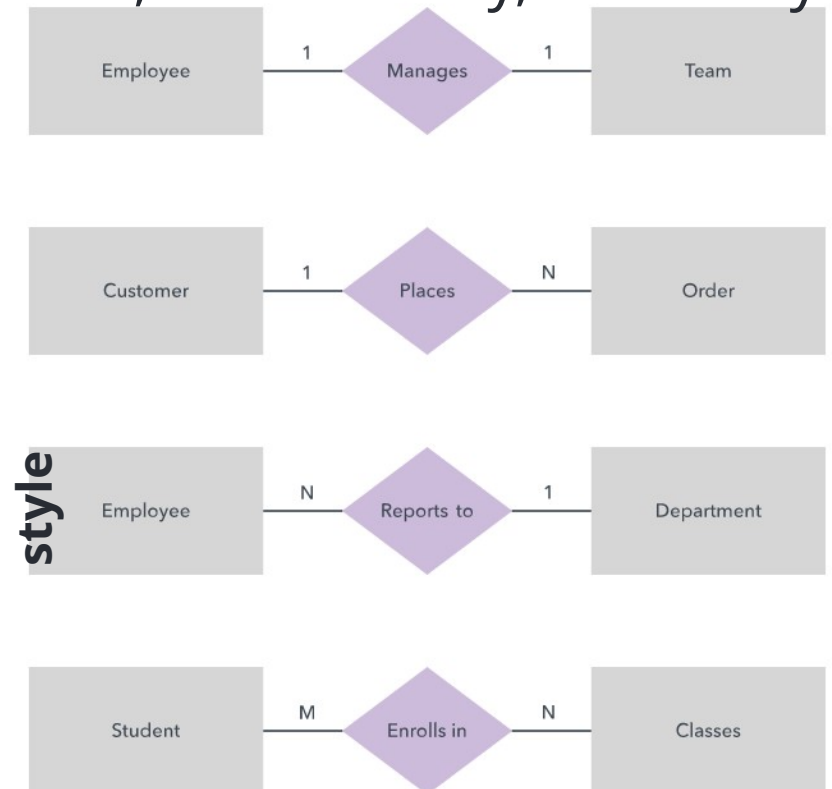
ENTITY RELATION DIAGRAM

- **Cardinality:** Defines the numerical attributes of the relationship between two entities or entity sets.
- The three main cardinal relationships are one-to-one, one-to-many, and many-many.



Crow's Foot/Martin/Information Engineering style

Chen notation style



ENTITY-RELATIONSHIP DIAGRAMS

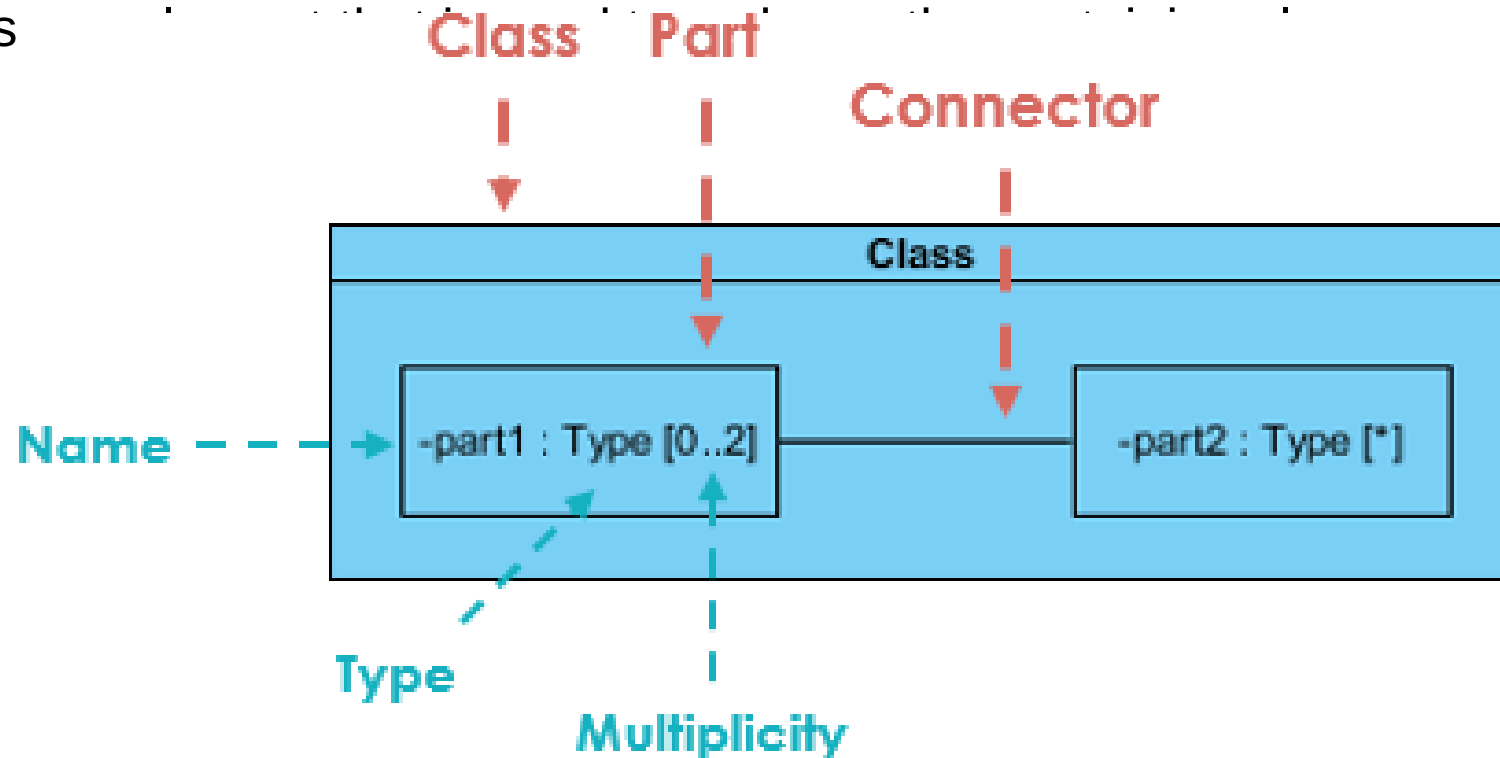
- ER diagrams are popular because
 - they provide an overview of the problem to be addressed
 - the view is relatively stable when changes are made to the problem's requirements
- ER diagram is likely to be used to model a problem early in the requirements process

Composite structure Diagram

- A composite structure diagram is a UML structural diagram that contains classes, interfaces, packages, and their relationships, and that provides a logical view of all, or part of a software system.
- It shows the internal structure (including parts and connectors) of a structured classifier or collaboration.

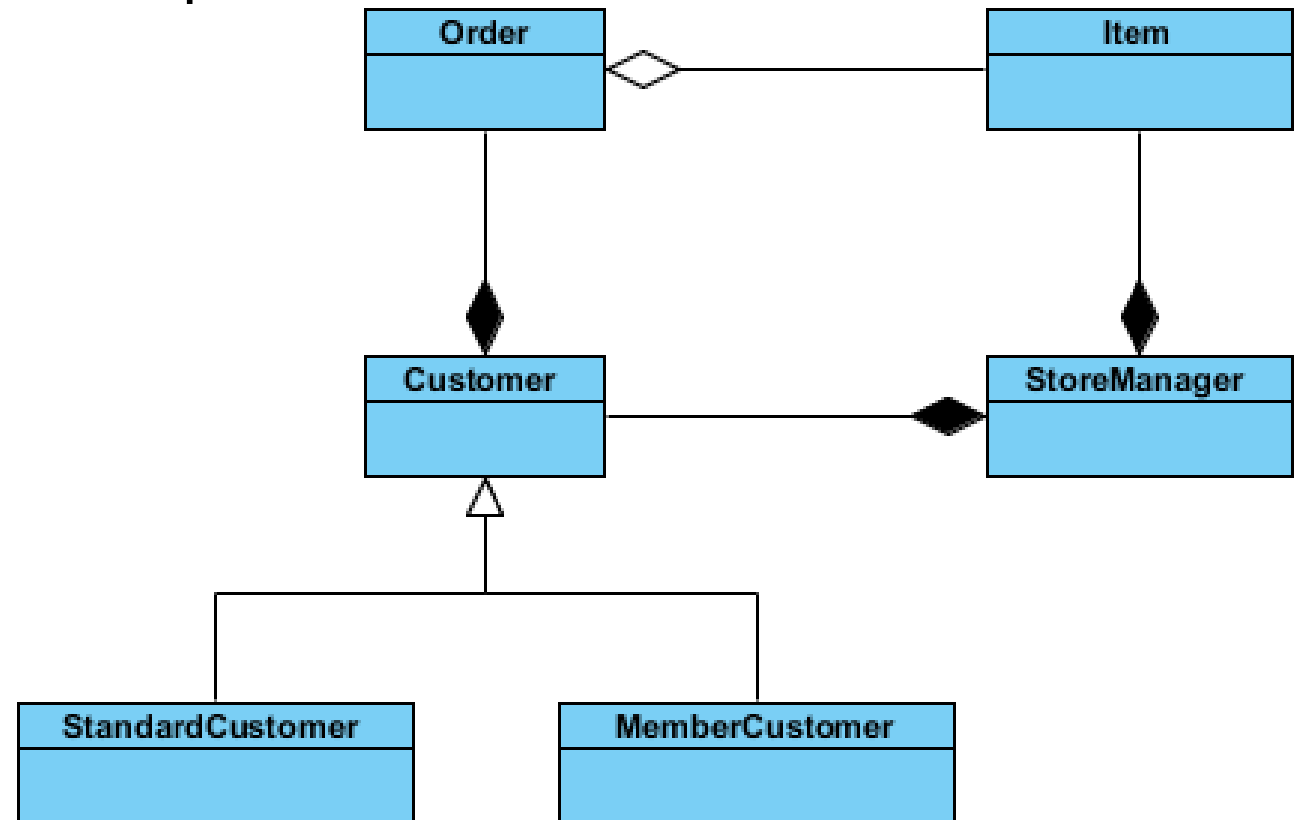
Composite structure Diagram

- Composite Structure Diagrams show the internal parts of a class.
- Parts are named: `partName:partType[multiplicity]`
- Aggregated classes are parts of a class but parts are not necessarily classes,
 - a part is

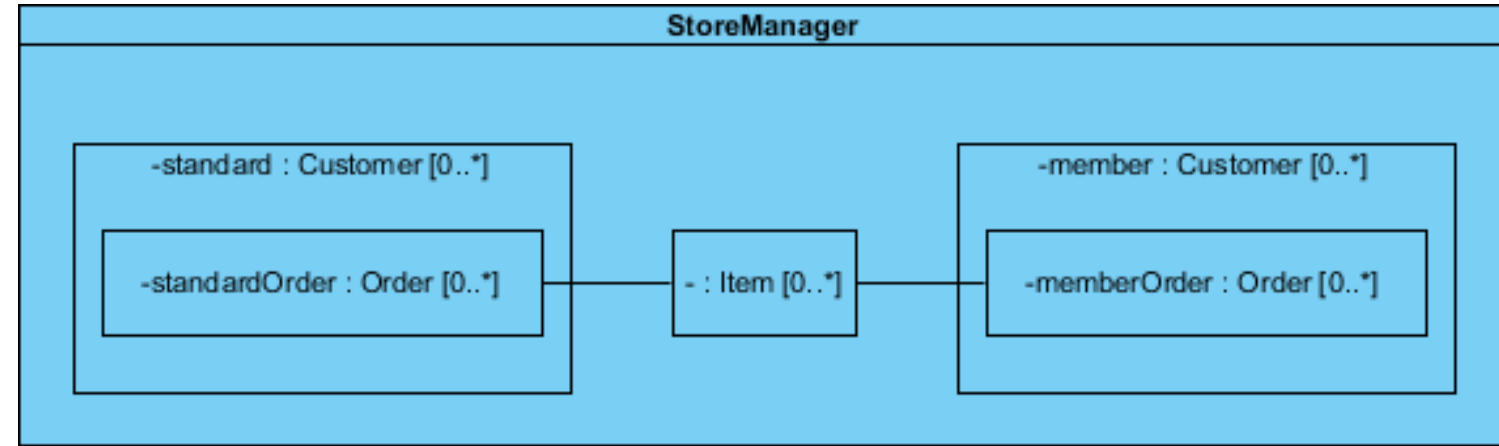
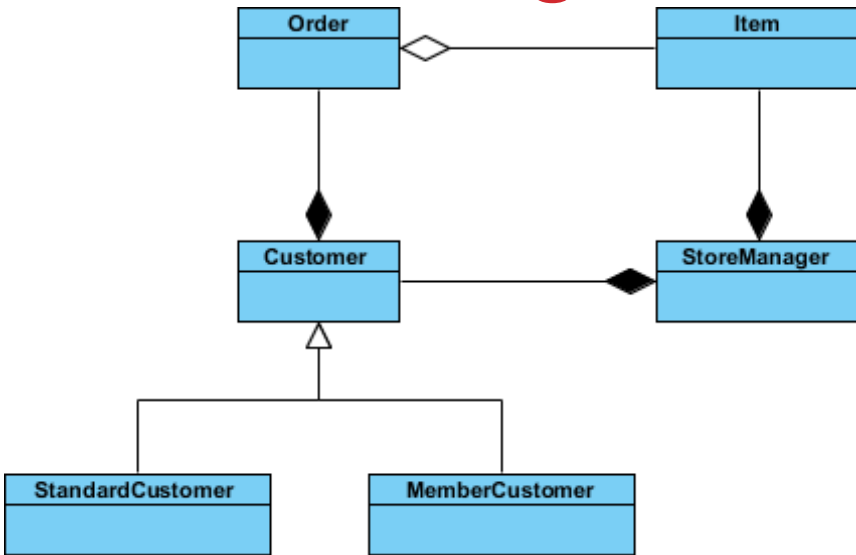


Class Diagram to Composite structure Diagram

- Example: Suppose we are modeling a system for an online store. The client has told us that customers may join a membership program which will provide them with special offers and discounted shipping, so we have extended the customer object to provide a member and standard option.
- We have a class for Item which may be aggregated by the Order class, which is composed by the Customer class which itself is composed by the StoreManager class. We have a lot of objects that end up within other objects.
- Everything looks like it ends up inside StoreManager, so we can create a composite structure diagram to really see what it's made of.



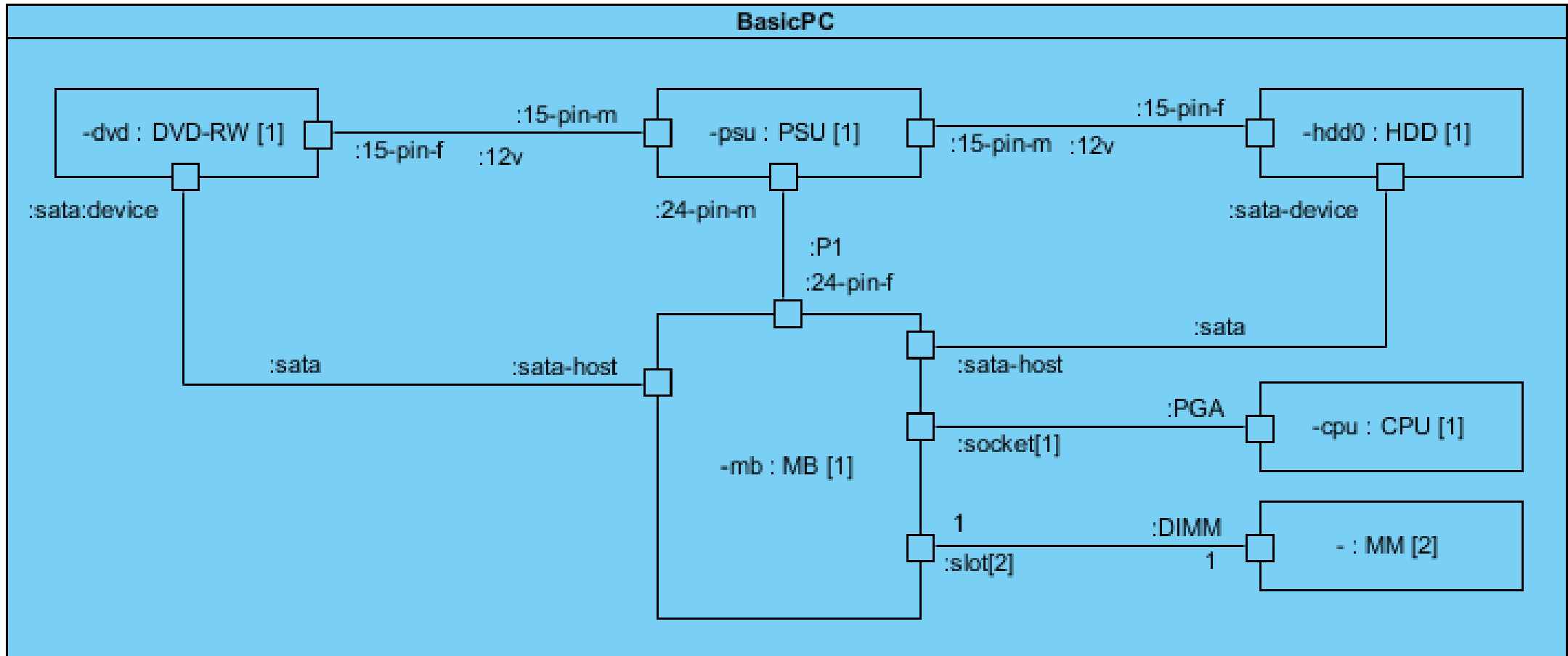
Class Diagram to Composite structure Diagram



- In the example above, we can see:
 - StoreManager from its own perspective, instead of the system as a whole.
 - StoreManager directly contains two types of objects (Customer and Item) as is indicated by the two composition arrows on the class diagram.
 - The composite structure diagram here shows more explicitly is the inclusion of the subtypes of Customer.
 - Notice that the type of both of these parts is Customer, as the store sees both as Customer objects.
 - We also see a connector which shows the relation between Item and Order.
 - Order is not directly contained within the StoreManager class but we can show relations to parts nested within the objects it aggregates.

Composite Structure Diagram

- Example - Computer System

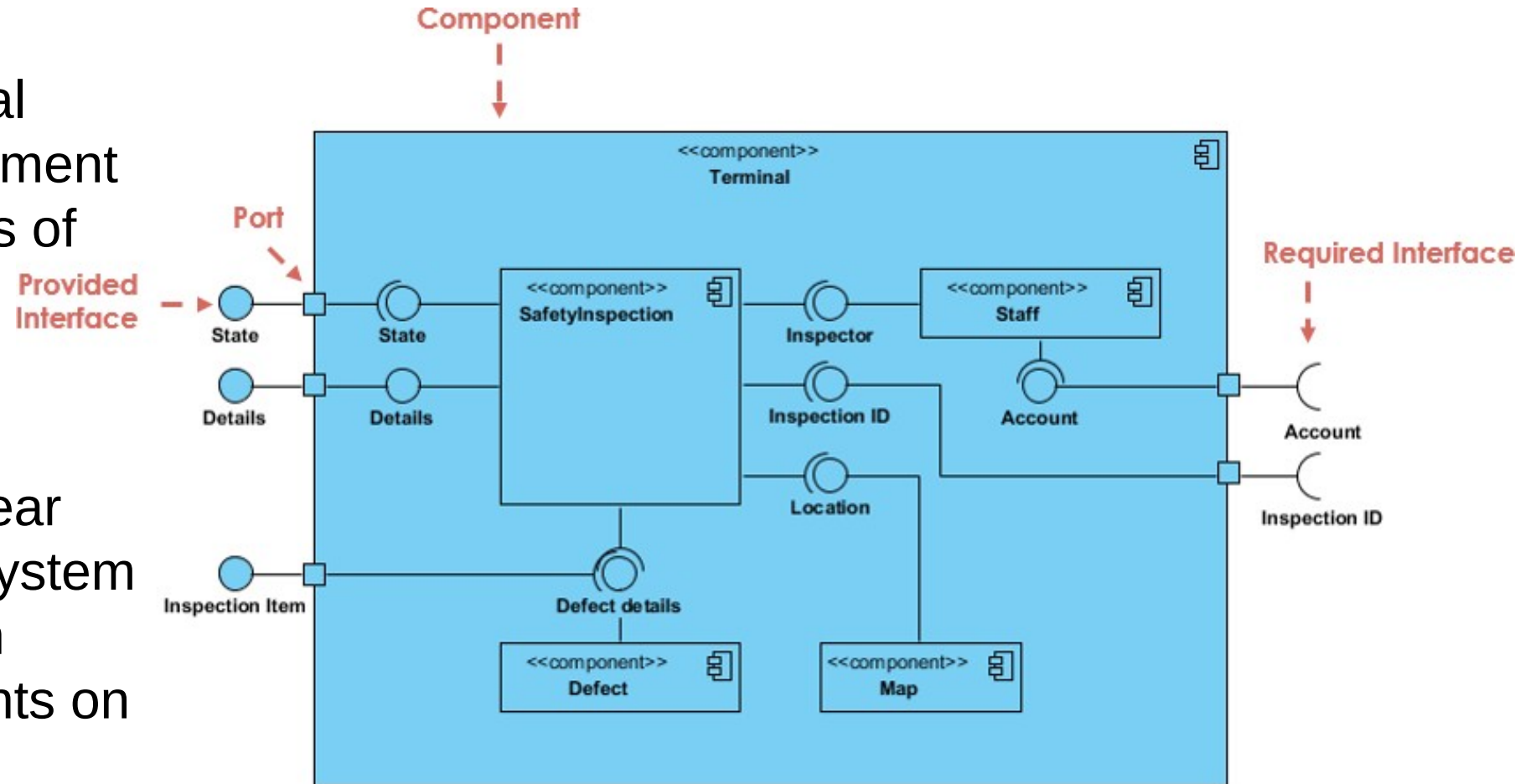


Component Diagram

- The purpose of a component diagram is to show the relationship between different components in a system.
- Component diagrams are essentially class diagrams that focus on a system's components that often used to model the static implementation view of a system.
- The term "component" refers to a module of classes that represent independent systems or subsystems with the ability to interface with the rest of the system.
- There exists a whole development approach that revolves around components: component-based development (CBD).
 - In this approach, component diagrams allow the planner to identify the different components so the whole system does what it's supposed to do.

Component Diagram at a Glance

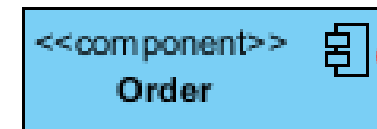
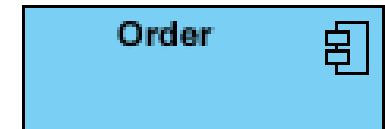
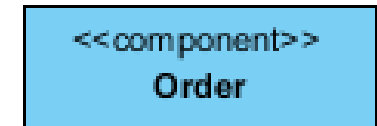
- A component diagram breaks down the actual system under development into various high levels of functionality.
- Each component is responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.



Basic Concepts of Component Diagram

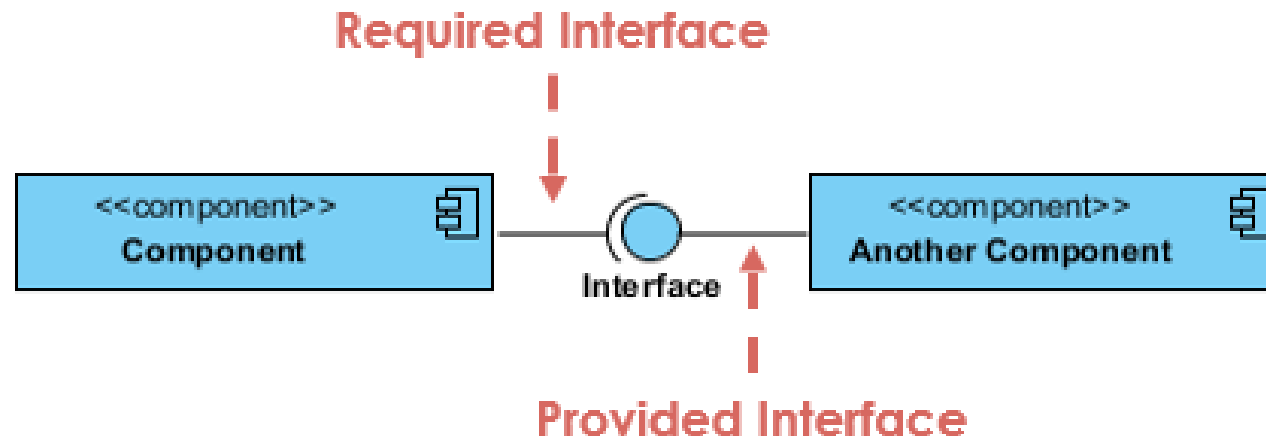
- In UML 2, a **component** is drawn as a rectangle with optional compartments stacked vertically. A high-level, abstracted view of a component in UML 2 can be modeled as:

1. A rectangle with the component's name
2. A rectangle with the component icon
3. A rectangle with the stereotype text and/or icon



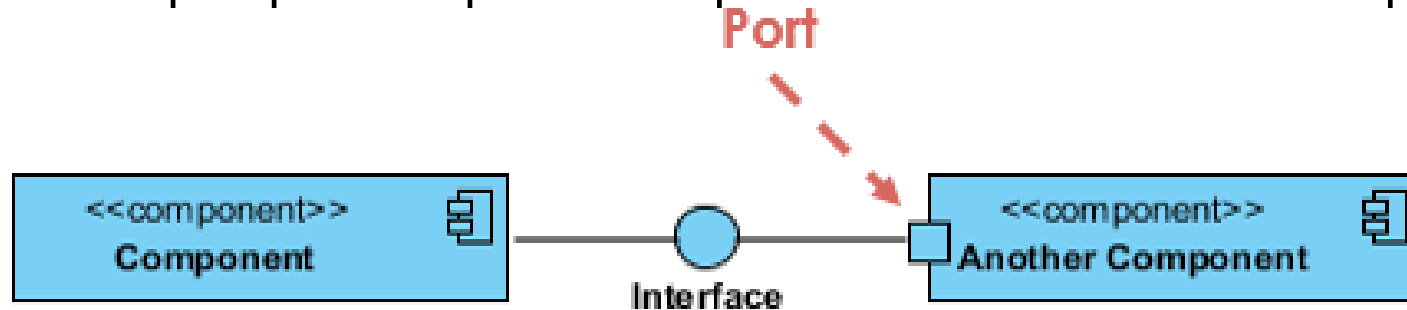
Basic Concepts of Component Diagram

- Interface:
- **Provided interface** symbols with a complete circle at their end represent an interface that the component provides - this "lollipop" symbol is shorthand for a realization relationship of an interface classifier.
- **Required Interface** symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires (in both cases, the interface's name is placed near the interface symbol itself).

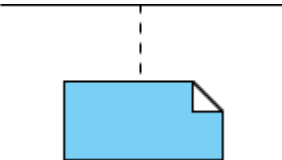
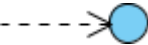


Basic Concepts of Component Diagram

- Port:
- **Ports** are represented using a square along the edge of the system or a component. A port is often used to help expose required and provided interfaces of a component.

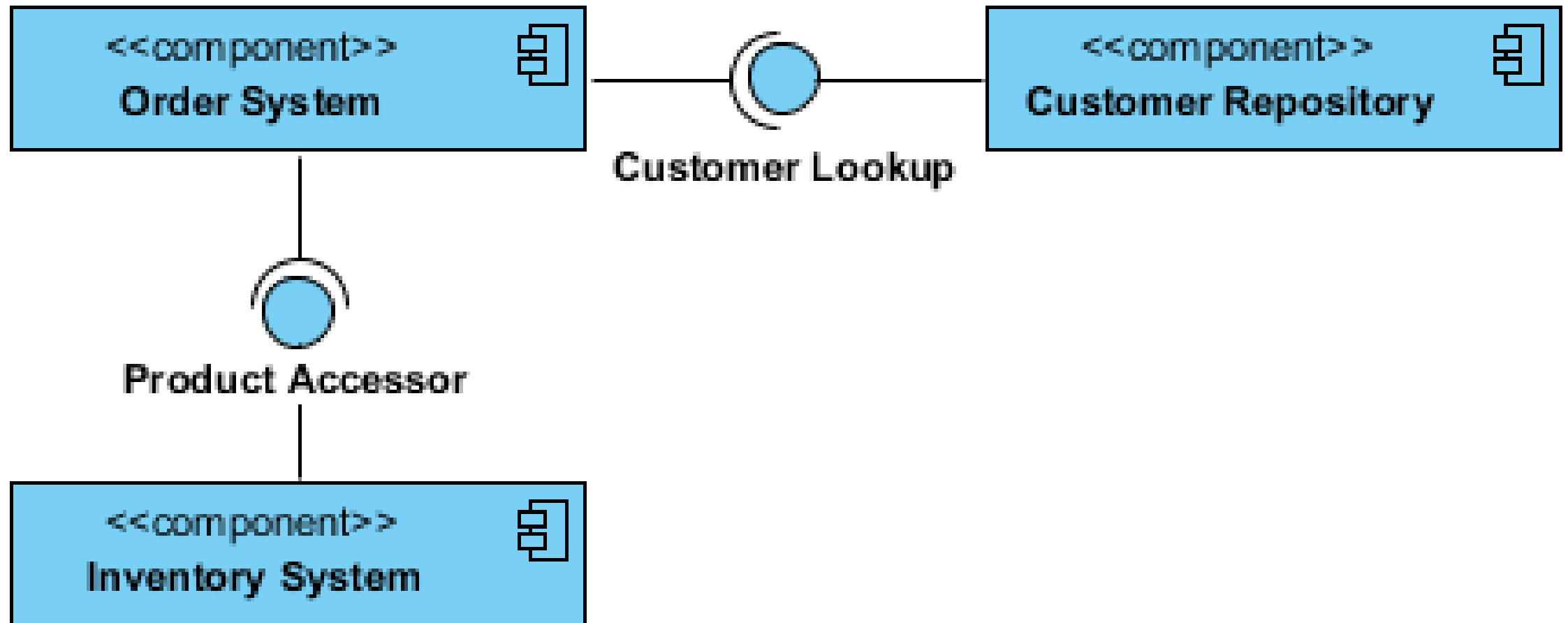


Basic Relationships of Component Diagram

Relation	Notation
Association	
Composition	
Aggregation	
Constraint	
Dependency (Lollipop)	
Links	

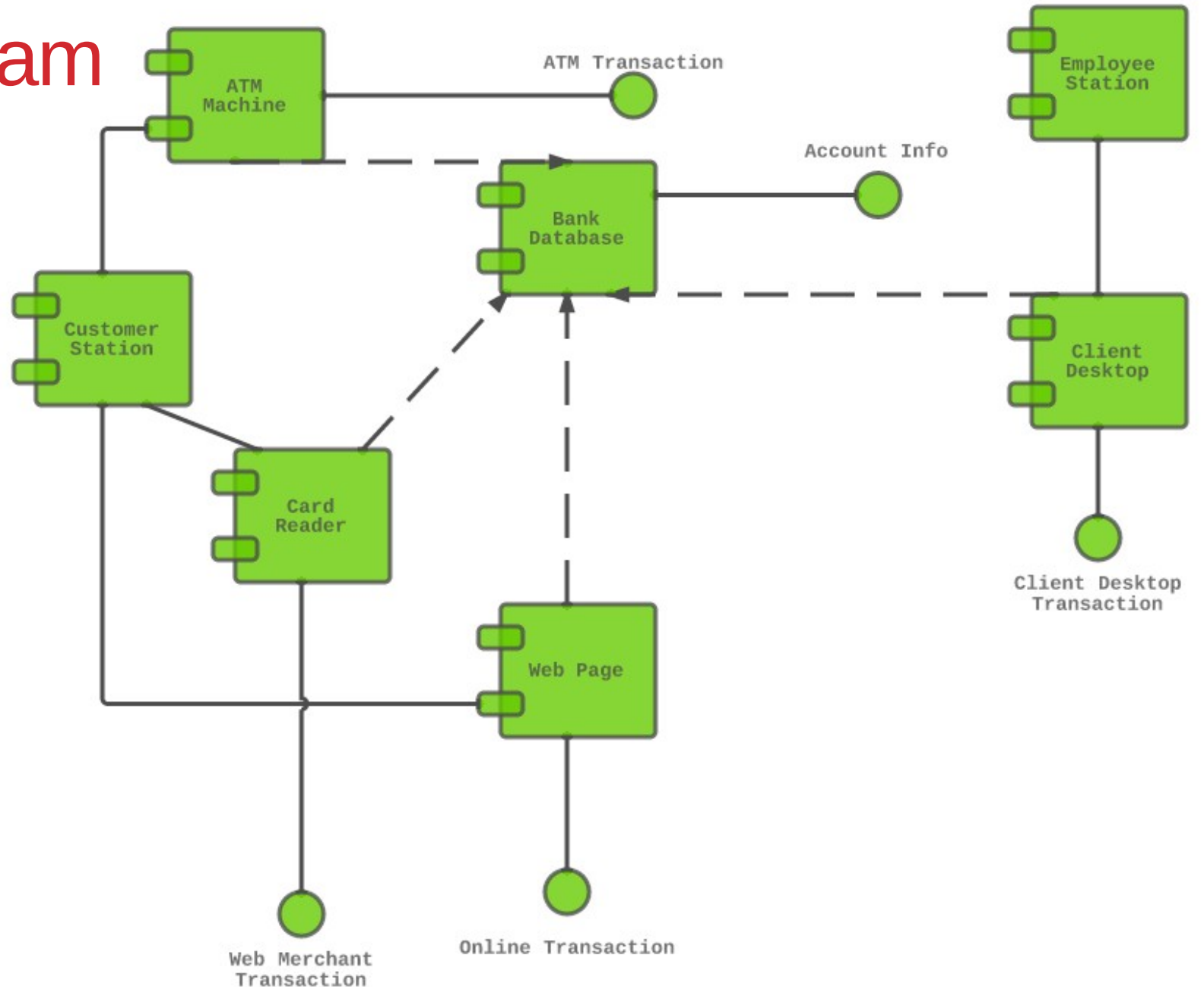
Component Diagram

- Example: Using Interface (Order System)



Component Diagram

- Example: ATM System

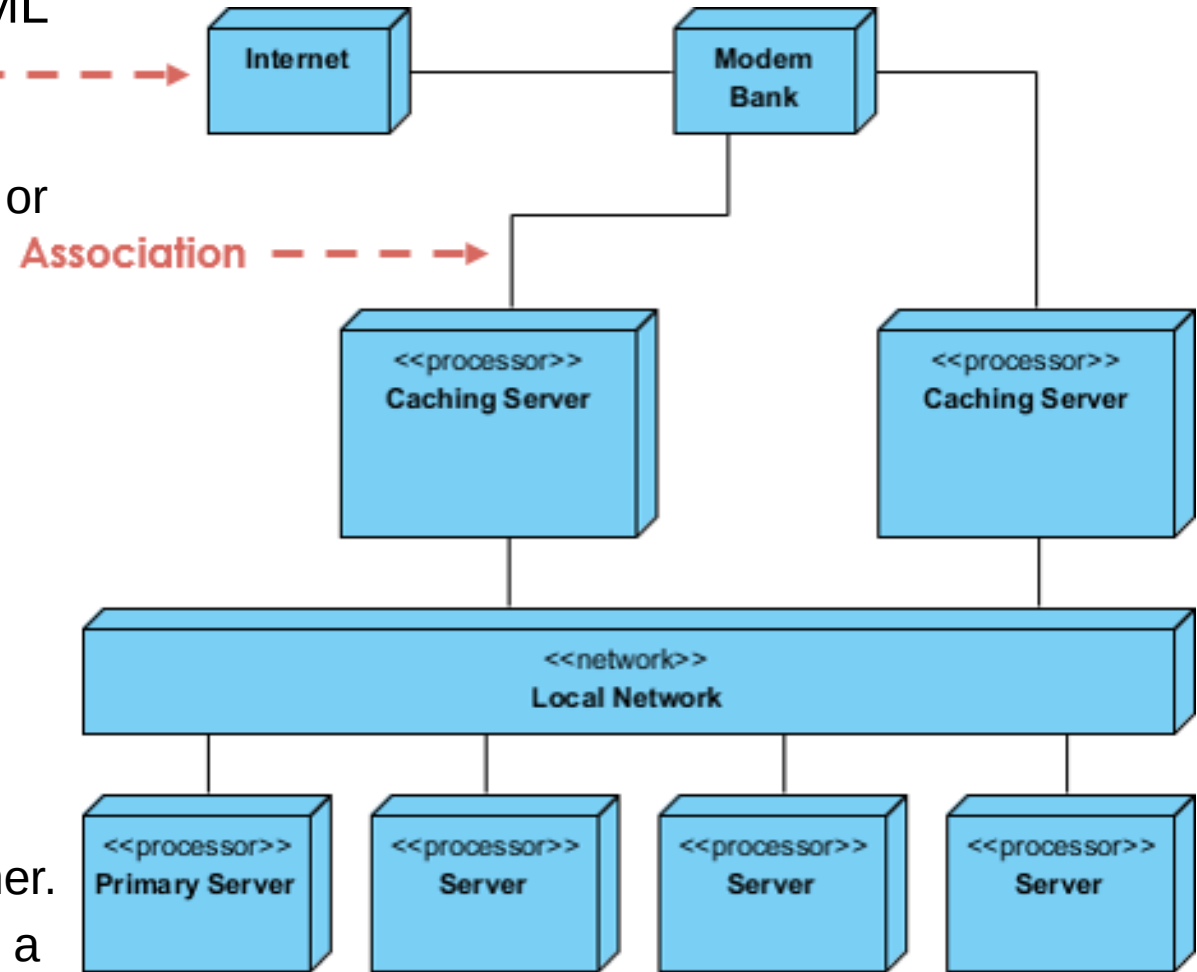


Deployment Diagram

- A UML deployment diagram shows the configuration of run time processing nodes and the components that live on them.
- Deployment diagrams is a kind of structure diagram used in modeling the physical aspects of an object-oriented system.
- The deployment diagram describes the physical deployment of information generated by the software program on hardware components.
- The information that the software generates is called an artifact.

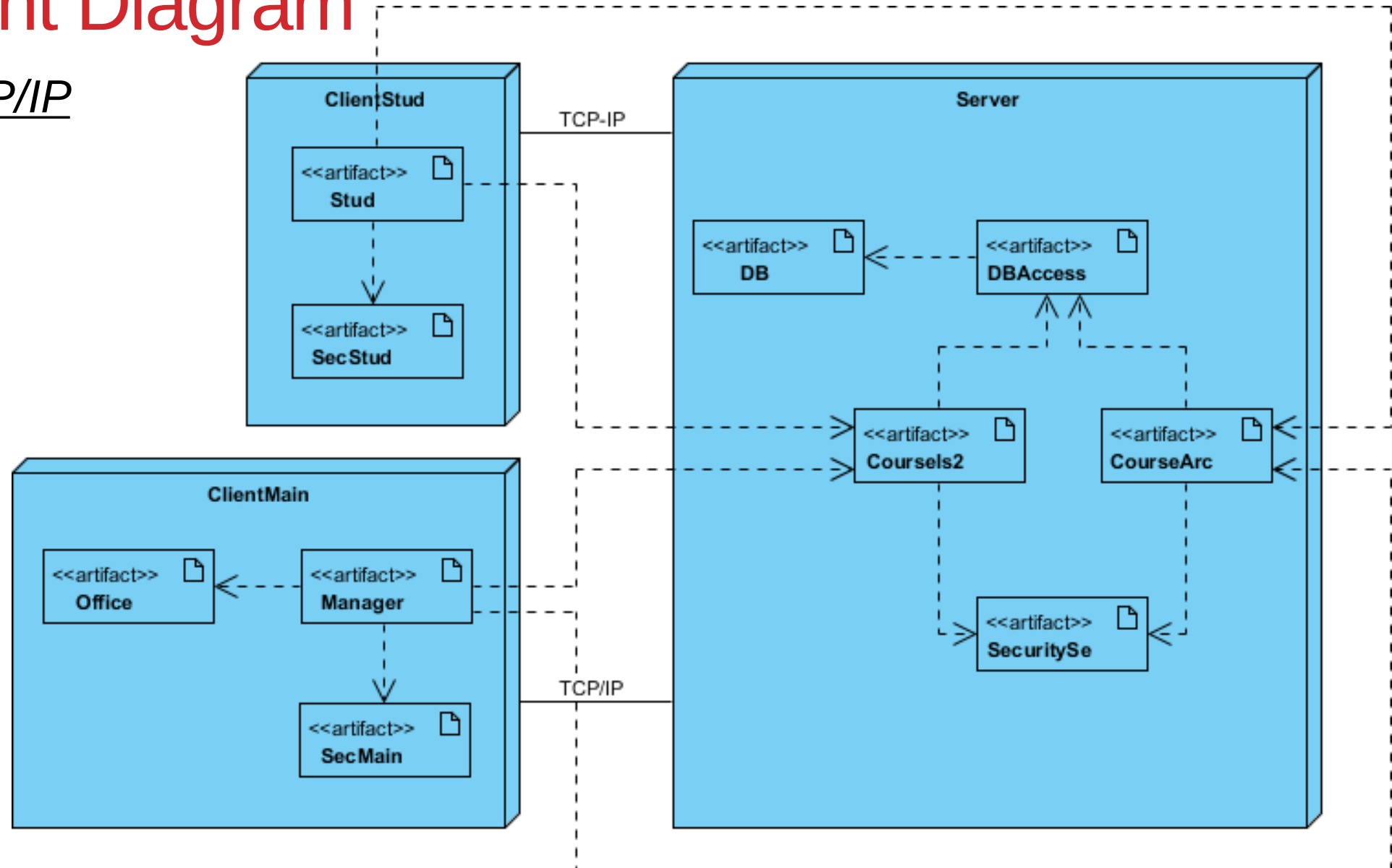
Deployment Diagram Basic Components

- Deployment diagrams are made up of several UML shapes.
 - The three-dimensional boxes, known as nodes, represent the basic software or hardware elements, or nodes, in the system.
 - HW node can be signified with <<stereotype>>,
 - Stereotype: A device contained within the node, presented at the top of the node, with the name bracketed by double arrows.
 - Connections between nodes are represented with a line, with optional <<stereotype>>
 - The smaller shapes contained within the boxes represent the software artifacts that are deployed.
 - A dashed line that ends in an arrow, which indicates that one node or component is **dependent** on another.
 - Component: A rectangle with two tabs that indicates a software element.



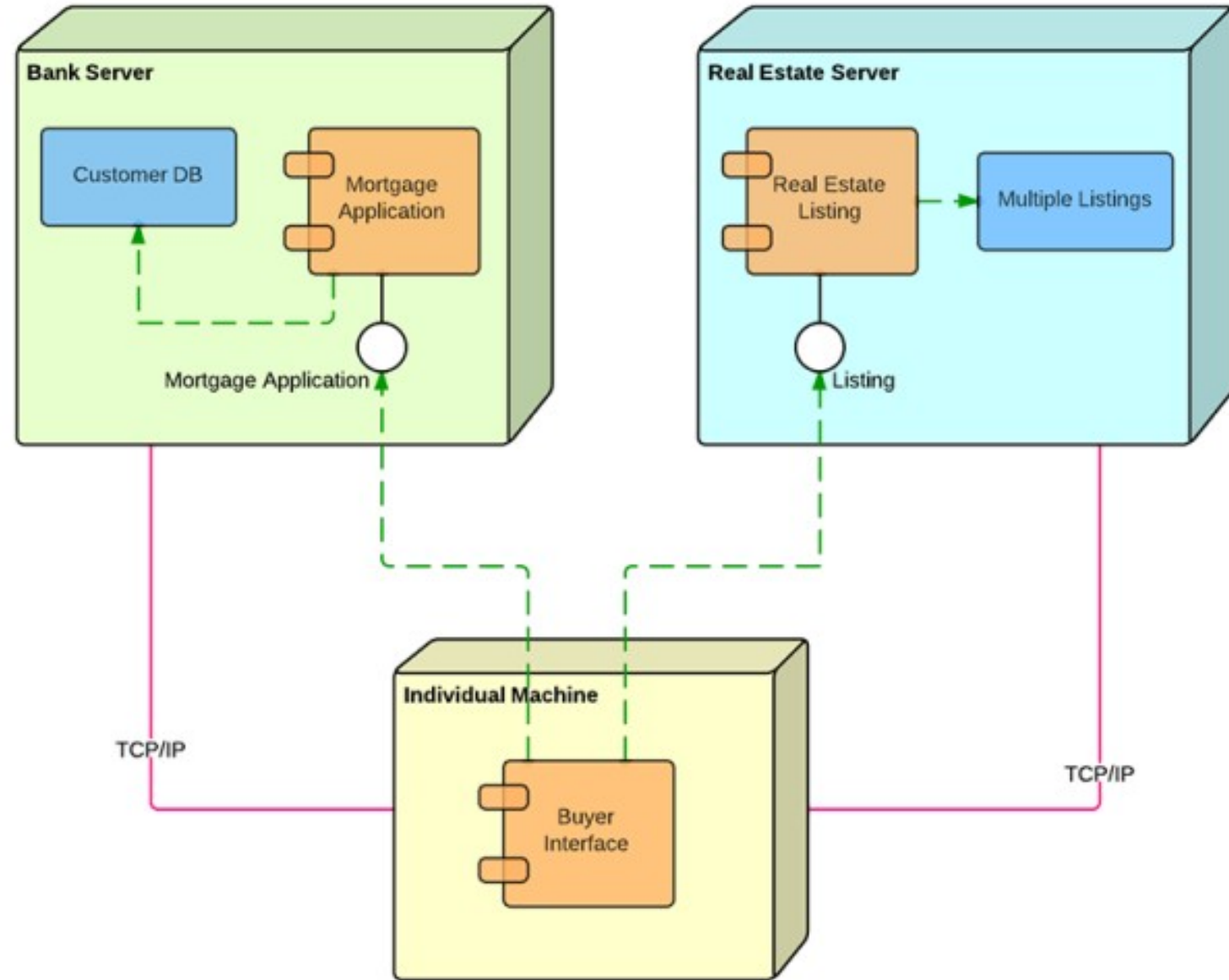
Deployment Diagram

- Example1: TCP/IP
Client / Server



Deployment Diagram

- Example2: TCP/IP
Client / Server

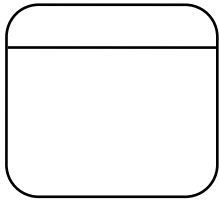


DATA FLOW DIAGRAMS (DFD)

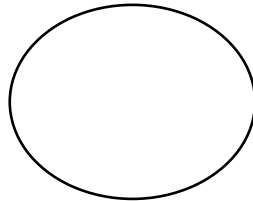
- ER diagram, event trace, state machines depict only lower-level details
- A data-flow diagram (DFD) models functionality and the flow of data from one function to another
 - A bubble represents a *process*
 - An arrow represents *data flow*
 - A *data store*: a formal repository or database of information
 - Rectangles represent *actors*: entities that provide input data or receive the output result

DFD SYMBOLS AND DEFINITIONS

**Process
or
Bubble**



or



- Process - performs some action on data, such as creates, modifies, stores, delete, etc. Can be manual or supported by computer.

Data store

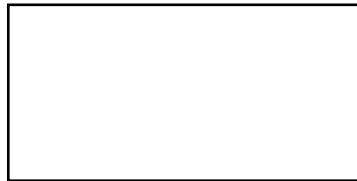


or



- Data store - information that is kept and accessed. May be in paper file folder or a database.

**Actors
or
External Entity**



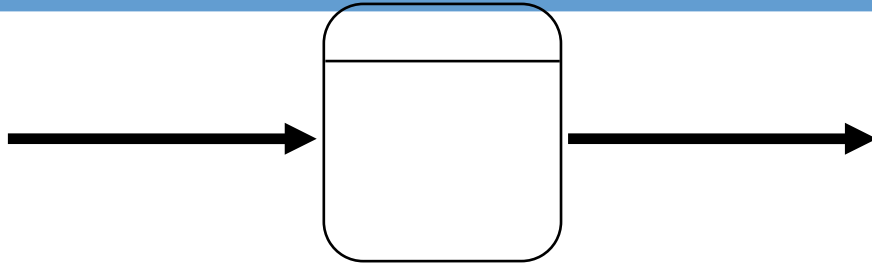
- External entity - is the origin or destination of data. Entities are external to the system.

Data flow

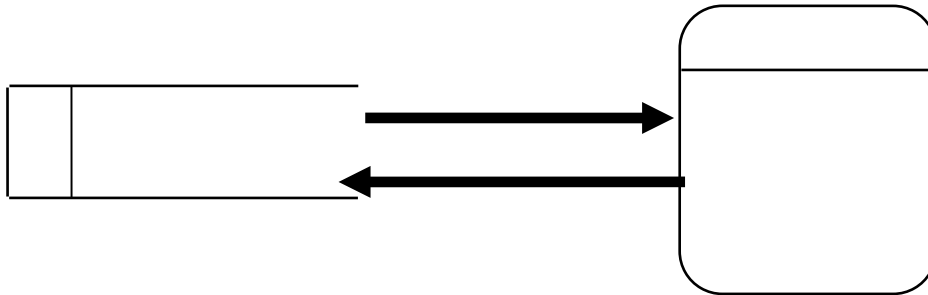


- Data flow - the flow of data into or out of a process, datastore or entity

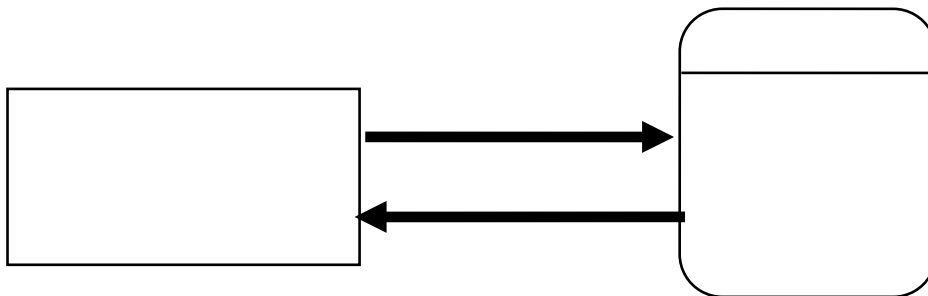
RULES FOR DRAWING DFD



A **minimum** of one data flow in and one data flow out of a process



A datastore **must** be connected to a process (either in, out, or both)

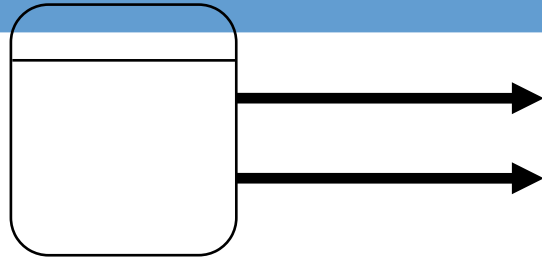


An external entity **must** be connected to a process (either in, out, or both)

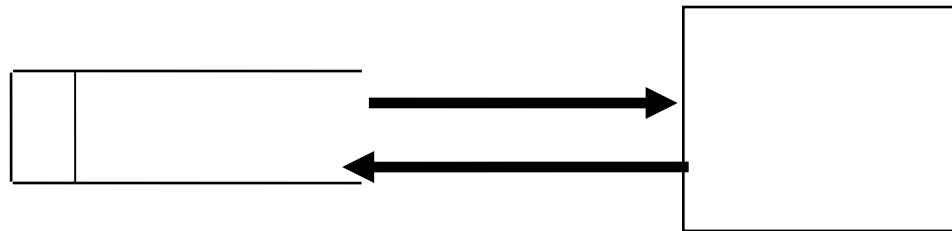


A single data flow **must** only flow one way

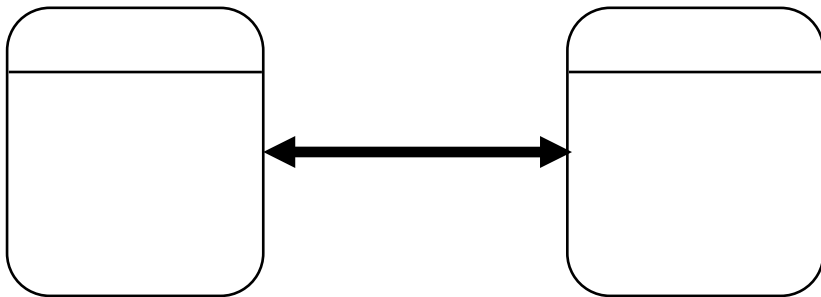
DFD: COMMON MISTAKES



- Process has no data flowing into it, but has data flowing out.



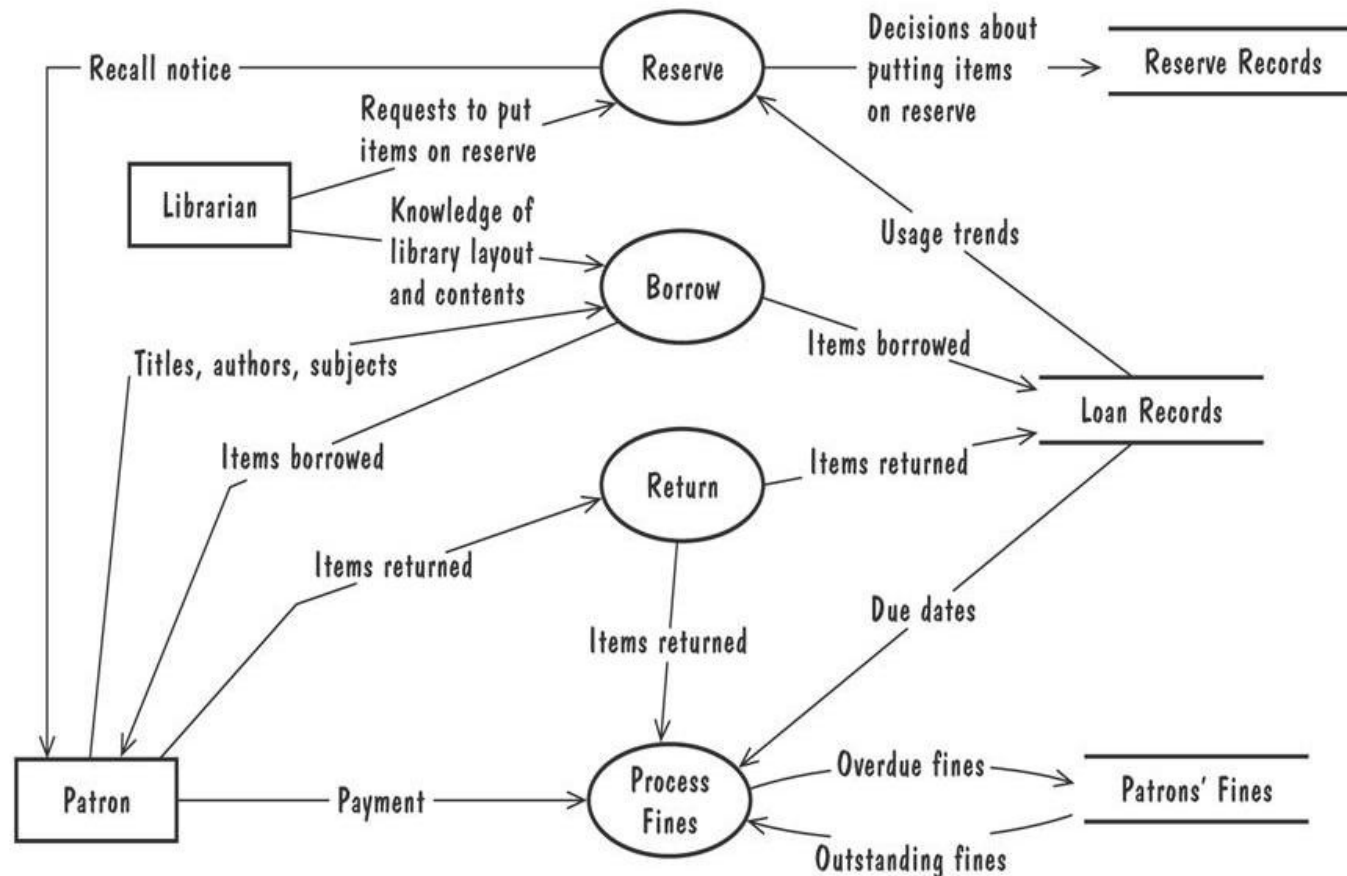
- Data store is hooked to external entity. This means external entity can read and write to your data file without auditing!!



- The data flow goes in two directions at once. Two or more arrows should be used to show the flow to and from each process.

DATA FLOW DIAGRAMS (CONTD.)

- A high-level data-flow diagram for the library problem

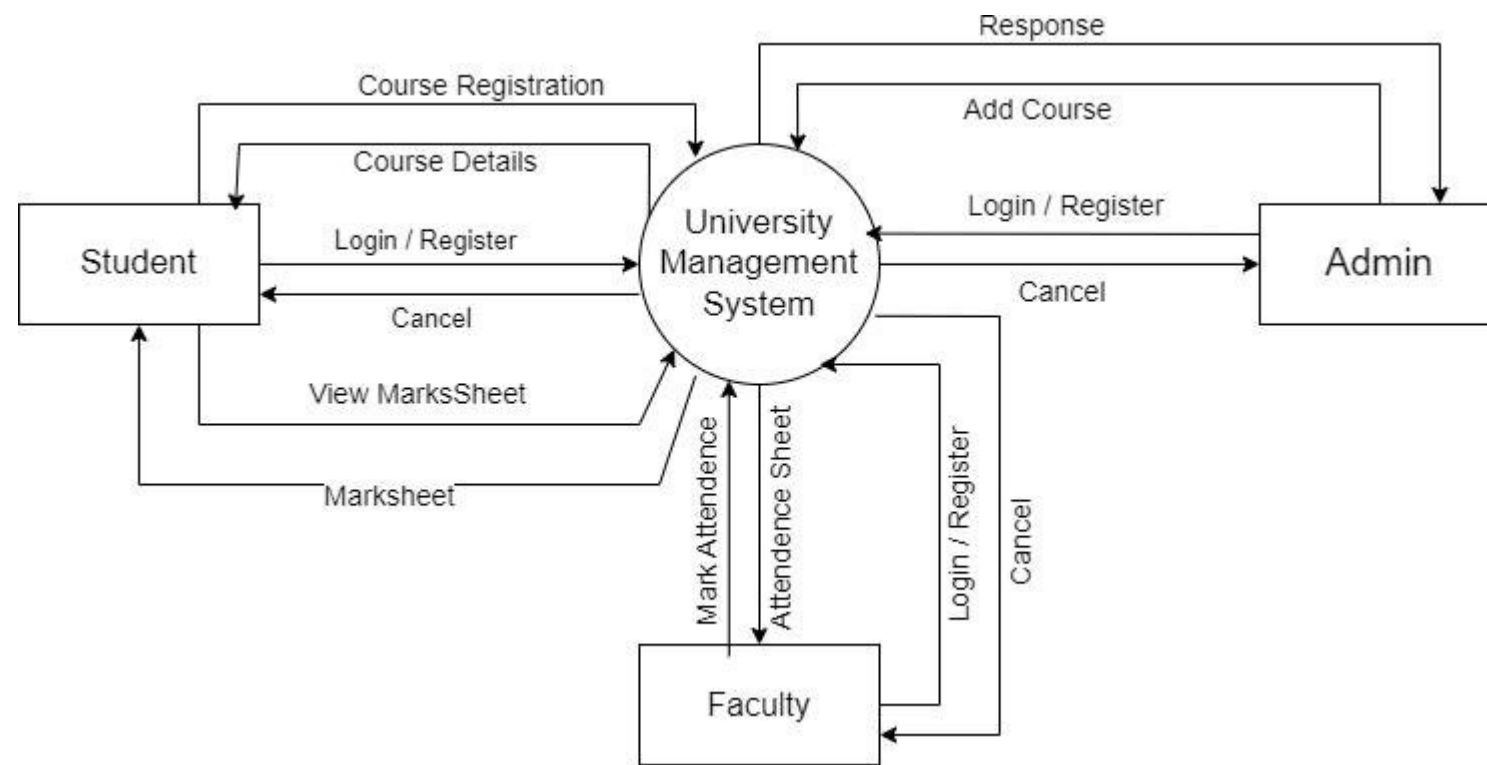


DFD: ADDING LEVELS OF DETAIL

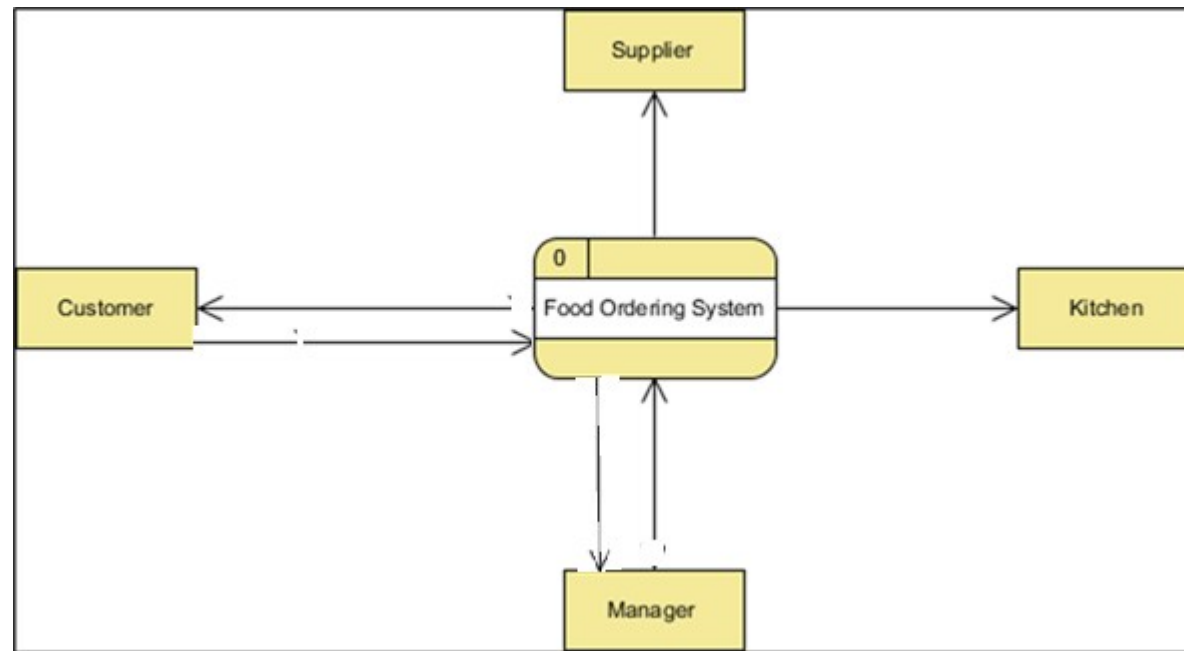
- The highest level, called the *context diagram* (also known as *level 0*), is only an overview. More detail is typically needed for system analysts.
- We add detail to a DFD by creating “levels”. The first level added after the context diagram is called level “1”.
- Each new level breaks apart one process and “decomposes” the single process into a new, more detailed DFD. A complete DFD can have many (up to 6 or 7) levels depending on the complexity of system.
- Breaking the DFD into levels is referred to as **“Decomposition”**.

DFD: NUMBERING LEVELS

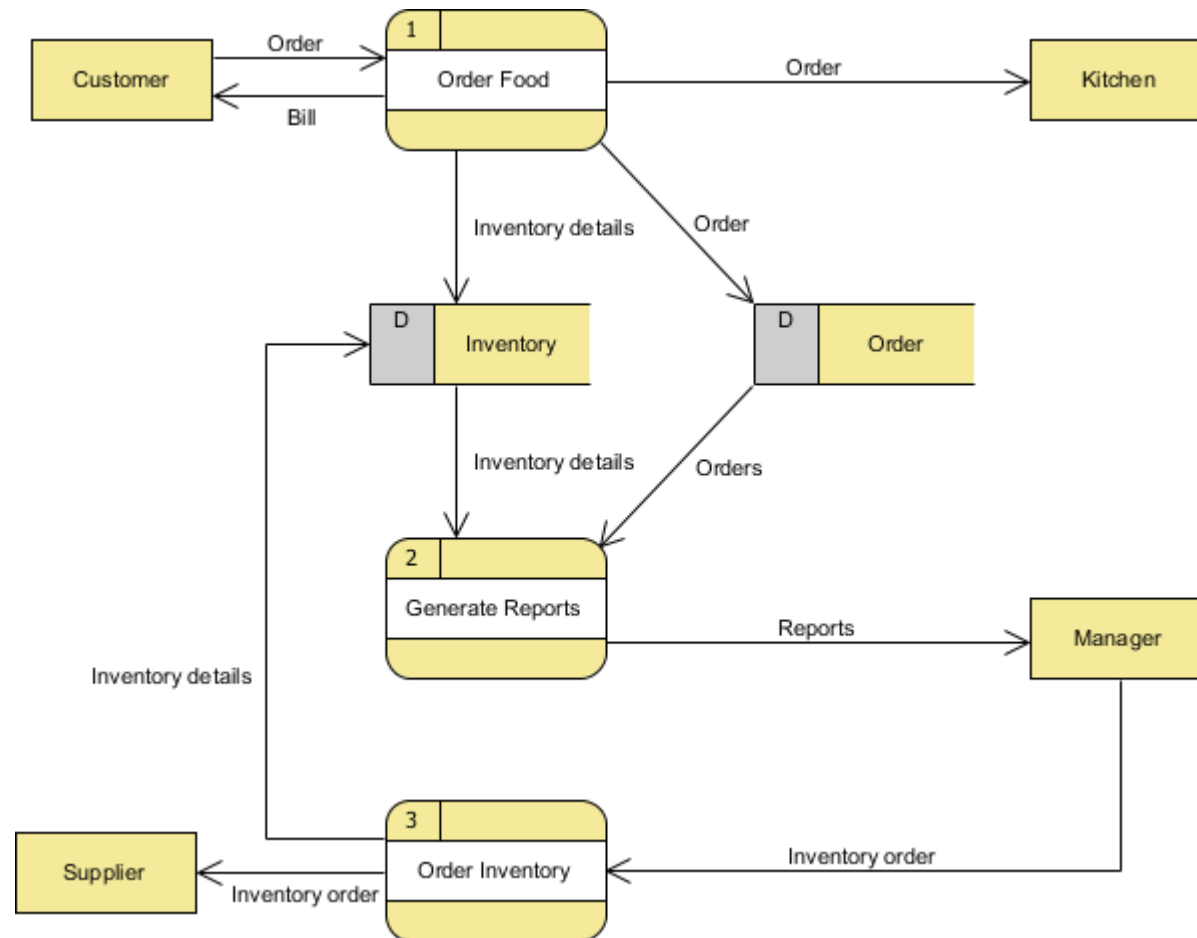
- In a DFD with many levels it's easy to forget which level you are on. That's why each level has different numbering for the processes on the diagram. The 'level' corresponds to the number of decimal places required to define a process in it. Here's how it works:
 - Context Diagram (Level 0) Process labeled "0"
 - Level 1 Processes labeled 1.0, 2.0, 3.0, .
 - Level 2 Processes labeled 1.1, 1.2, 1.3, .
 - Level 3 Processes labeled 1.11, 1.12,...



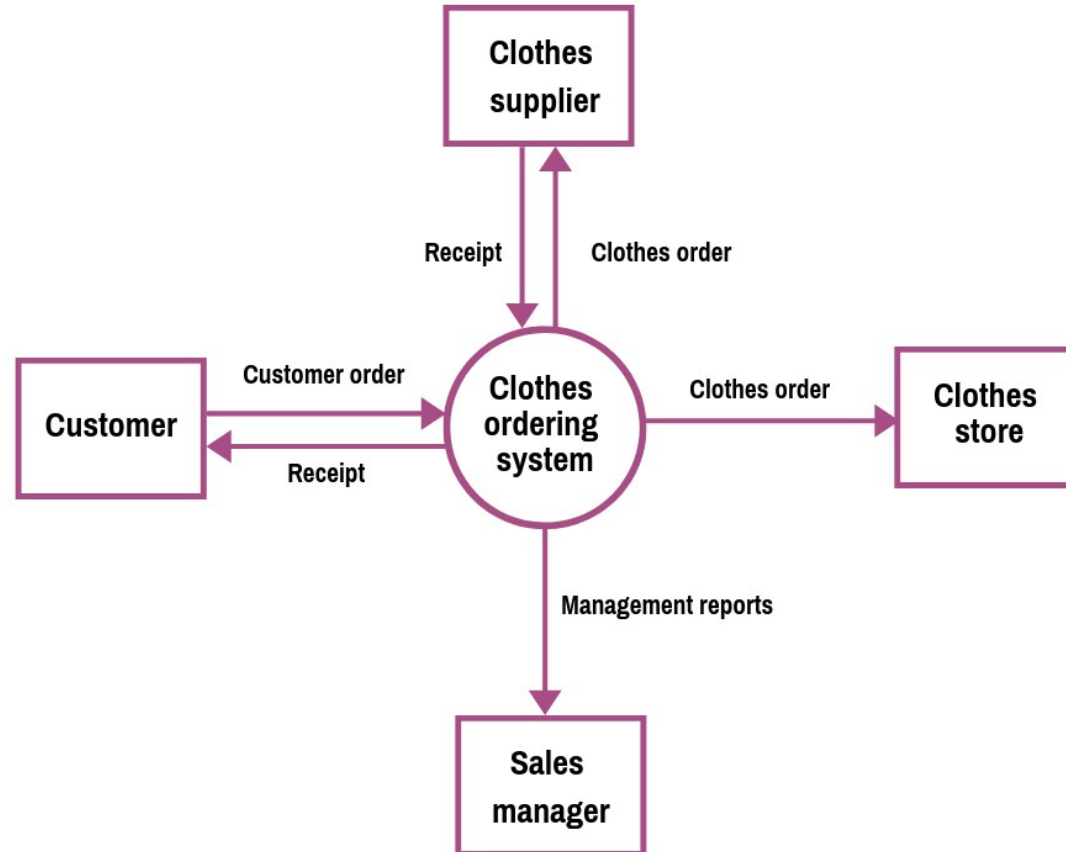
CONTEXT DFD



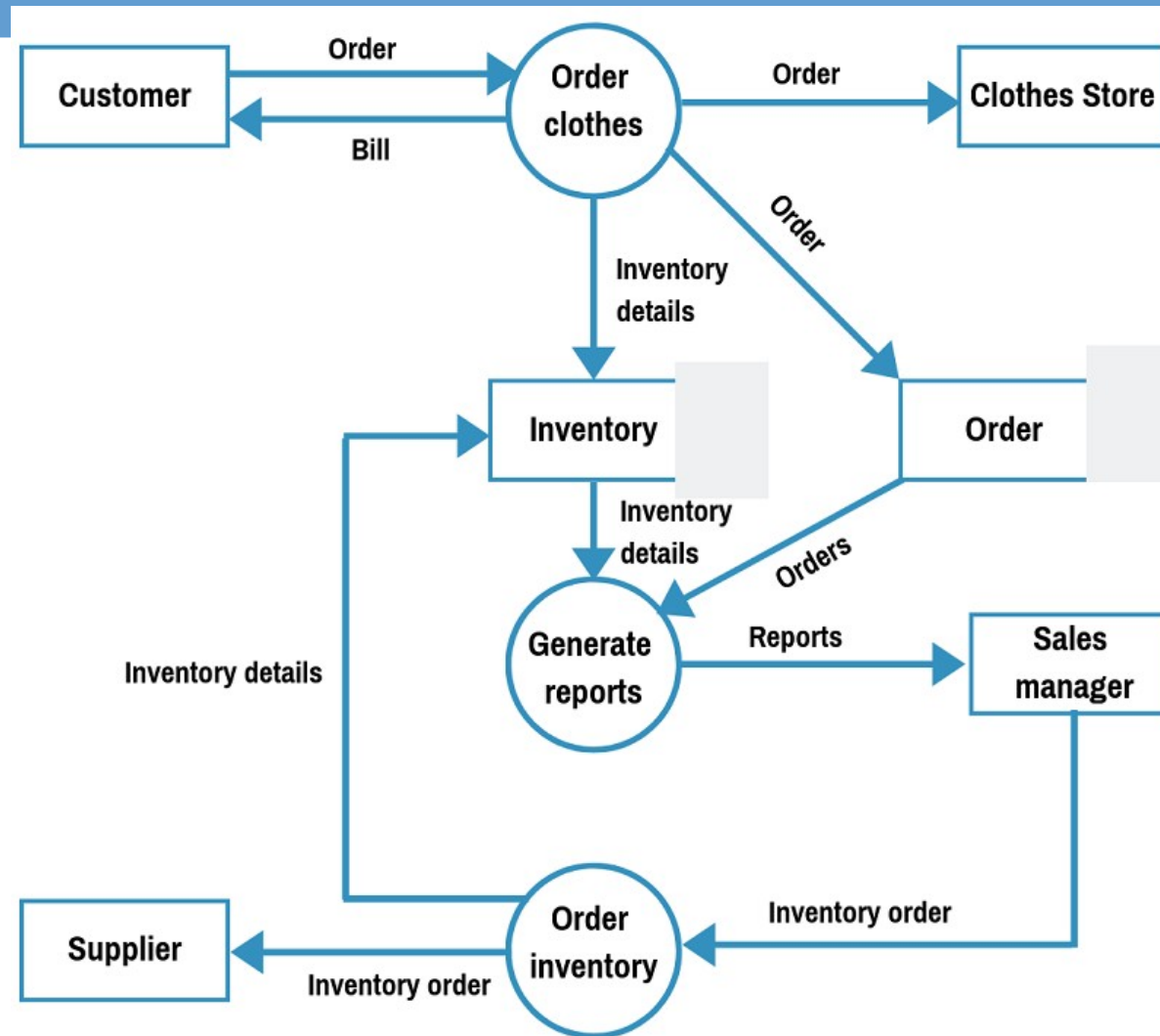
LEVEL 1 DFD



CONTEXT DFD

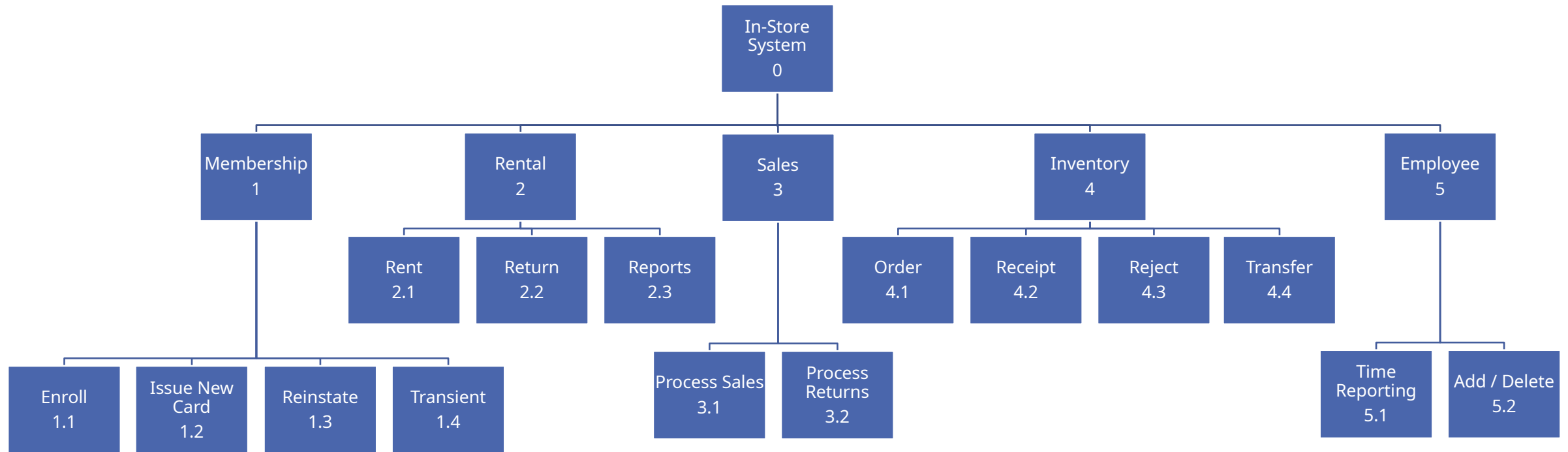


LEVEL 1 DFD



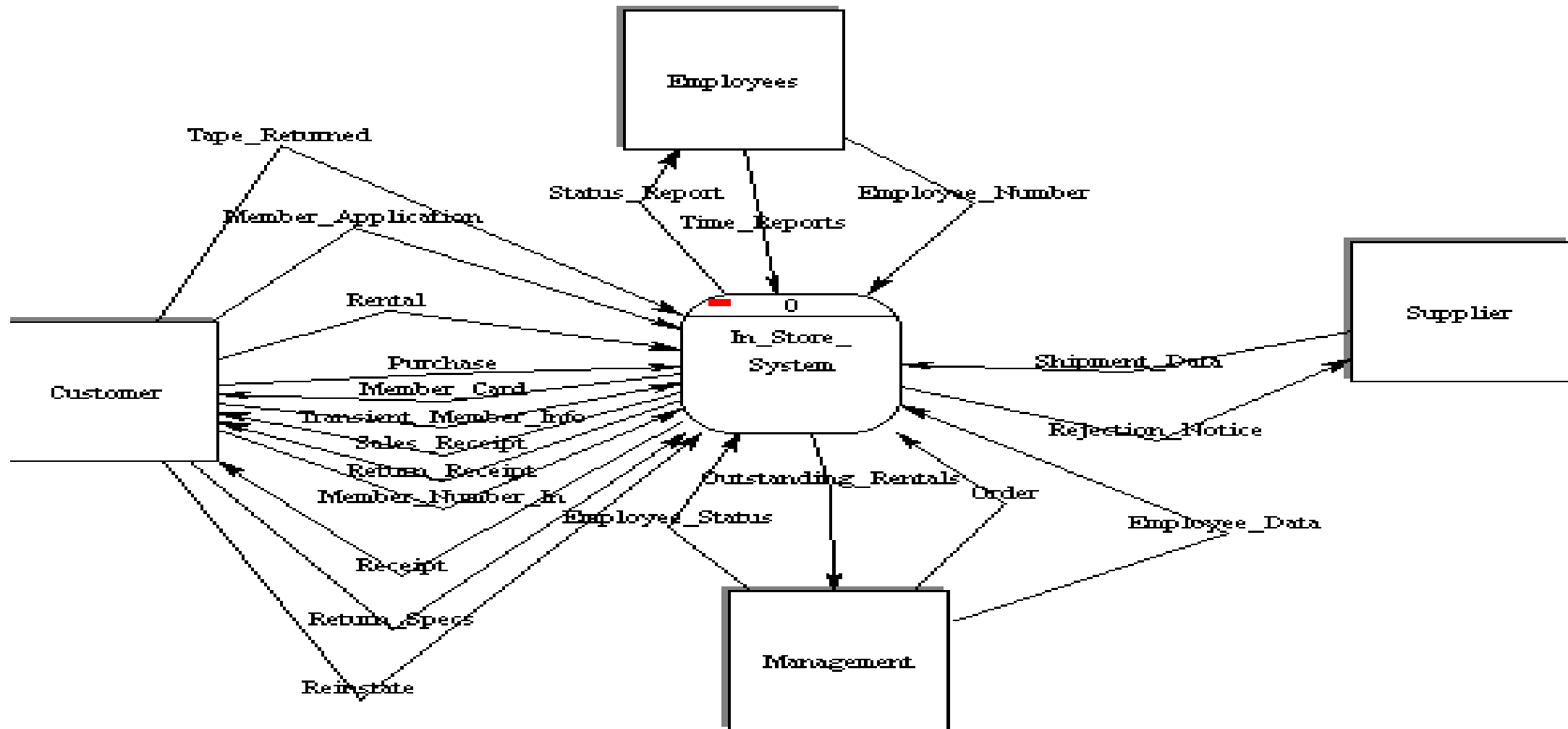
PROCESS HIERARCHY (WBS)

(THE BROADWAY ENTERTAINMENT COMPANY)



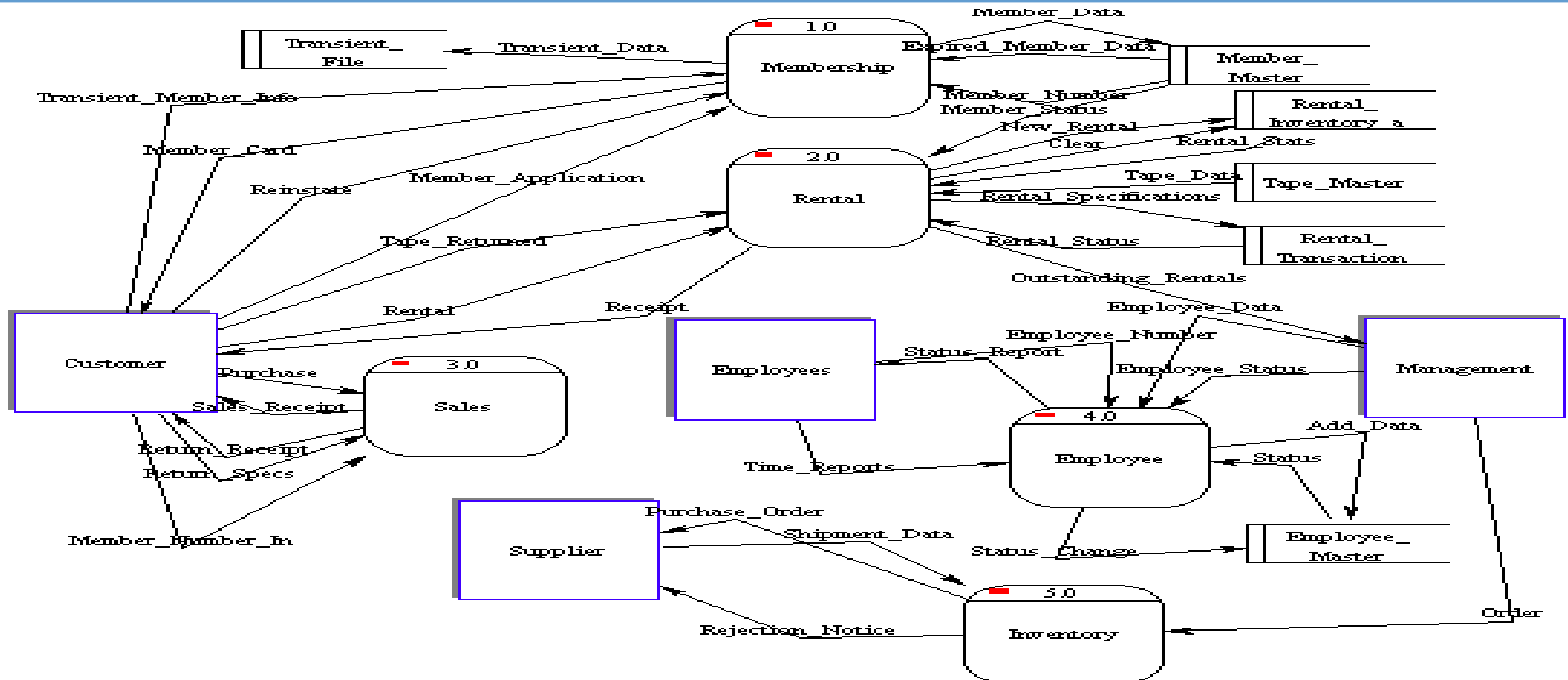
CONTEXT LEVEL (LEVEL 0)

(THE BROADWAY ENTERTAINMENT COMPANY)



LEVEL 1

(THE BROADWAY ENTERTAINMENT COMPANY)



DATA FLOW DIAGRAMS (CONTD.)

- Note that the Context Diagram did NOT record the movement of merchandise, but rather the information flow surrounding the underlying business processes.
 - A DFD that shows the movement of information is called a “**logical**” DFD.
 - A DFD that shows the movement of physical stuff (such as merchandise) is called a “**physical**” DFD.
- The **logical DFD** is by far the most common type of DFD and therefore we will focus on this in this class.

DFD: SOME FINAL NOTES

- How many processes on a diagram?
 - rule of thumb is to have 5 - 7 processes, if you need more, create a new level.
- Are there conventions for naming objects?
 - process names are verb phrases
 - all others are noun phrases
- How many levels should you go?
 - Only as many as you need to communicate adequately. The key is efficient communication.